



COMSATS University Islamabad, Pakistan

E-Garments “Forever”

A project presented to

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Bilal Sajid

CUI/SP22-BCS-070/SWL

Muhammad Sabtain

CUI/SP22-BCS-087/SWL

Usman Ali

CUI/SP22-BCS-117/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Bilal Sajid

Muhammad Sabtain

Usman Ali

CERTIFICATE OF APPROVAL

It is to certify that the final year project “E-Garment "Forever" was developed by **Muhammad Bilal Sajid (CUI/SP22-BCS-070/SWL)**, **Muhammad Sabtain (CUI/SP22-BCS-087/SWL)** and **Usman Ali (CUI/SP22-BCS-117/SWL)** under the supervision of MR. Tariq Rafiq and co-supervisor Dr. Javed Ferzund. In their opinion, it is fully adequate in scope and quality for the degree of Bachelor of Science in Computer Science.

Supervisor

Mr. Tariq Rafiq

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Muhammad Munawar Iqbal

Assistant Professor

Department of Computer Science

University of Engineering and Technology (UET), Taxila

Head of Department

Dr. Zafar Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

E-Garments Forever is a modern, responsive, and fully featured e-commerce website developed to enhance the online shopping experience for both customers and fashion retailers. The platform allows users to easily create accounts, browse a wide variety of men's and women's clothing and accessories, add products to their cart, place secure online orders, and track their purchase history. It aims to simplify the entire online shopping journey by providing a clean, user-friendly interface and fast, efficient functionality, ensuring a smooth experience from browsing to checkout.

For retailers, the system provides powerful tools for product management, category handling, stock updates, and order tracking. These built-in features help store owners manage their inventory digitally and reduce manual work, while improving overall operational efficiency. By offering an easy-to-manage dashboard, the platform supports small and medium-sized fashion businesses in expanding their online presence.

E-Garments Forever is built using the MongoDB, Express.js, React.js, and Node.js which ensures high scalability, performance, and security. React.js delivers a responsive, smooth, and interactive user interface, while Node.js and Express.js handle backend operations, API communication, and user authentication. MongoDB provides an efficient NoSQL database system capable of managing large product catalogs, user profiles, orders, and real-time updates. The website architecture is fully designed to support future enhancements such as payment integrations.

This document provides a comprehensive overview of E-Garments Forever, including system requirements, architecture design, implementation details, and testing procedures. The project is developed with a strong focus on usability, security, and reliability, ensuring that both customers and fashion retailers can benefit from a robust and feature-rich online shopping experience.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Tariq Rafiq and co-supervisor Dr. Javed Ferzund”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Bilal Sajid

Muhammad Sabtain

Usman Ali

Abbreviations

Abbreviation	Full Form
SRS	Software Requirement Specification
SDD	Software Design Document
API	Application Programming Interface
GPS	Global Positioning System
UX	User Experience
UI	User Interface
SDLC	Software Development Life Cycle
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
HCI	Human-Computer Interaction
VS Code	Visual Studio Code
UUID	Universally Unique Identifier
NoSQL	Non-Structured Query Language
CI	Continuous Integration
ReactJS	React JavaScript (Web Framework)
Node.js	Node JavaScript (Backend Runtime)
MongoDB	Mongo Database (NoSQL DBMS)
REST	Representational State Transfer
Expo Go	Framework for testing React Native apps
Postman	API Testing Tool

Table of Contents

1	Introduction.....	1
1.1	Brief	2
1.2	Relevance To Course Modules	3
1.3	Project Background.....	4
1.3.1	Problem Statement	5
1.3.2	Target Audience.....	5
1.3.3	Project Objectives	5
1.3.4	Technological Choices.....	6
1.3.5	System Features	6
1.3.6	Challenges and Innovations	7
1.4	Literature Review.....	7
1.4.1	OnlineFashion Platforms and Market Trends	7
1.4.2	Technologies for Building E-Commerce Platforms.....	8
1.4.3	Backend Development	8
1.4.4	Real-Time Order Status Tracking	8
1.4.5	Challenges in Online Fashion Platforms.....	9
1.4.6	Conclusion from Literature Review.....	9
1.5	Analysis from Literature Review	9
1.5.1	Real-Time Order Status Tracking	10
1.5.2	Scalable Backend Architecture	10
1.5.3	User Experience (UX) and User Interface (UI) Design	10
1.5.4	Real-Time Data Synchronization.....	11
1.5.5	Conclusion	11
1.6	Methodology and Software Lifecycle for This Project.....	11
1.6.1	Rationale behind Selected Methodology	12
2	Problem Definition.....	14
2.1	Problem Statement	15
2.2	Deliverables and Development Requirements	16
2.2.1	Key Deliverables.....	16
2.2.2	Development Requirements	16
2.3	Current System.....	17
2.3.1	Lack of Integration.....	17
2.3.2	User Experience Challenges	17
2.3.3	Manual Inventory and Product Management.....	17
3	Requirement Analysis.....	18
3.1	Use Cases Diagram	18
3.2	Detailed Use Case	24
3.3	Functional Requirements	38
3.3.1	Customer Module.....	38
3.3.2	Garments Module.....	39
3.4	Non-Functional Requirements	40
4	Design and Architecture.....	41
4.1	System Architecture.....	41
4.2	Data Representation [Diagram + Description].....	45
4.3	Process Flow/Representation	53

4.4	Design Models	57
4.4.1	Sequence Diagram	57
4.4.2	Class Diagram	61
5	Implementation	68
5.1	Backend Implementation	68
5.2	Customer Web Portal Implementation.....	69
5.3	User Interface.....	69
5.5.1	Web Interface (ReactJS)	69
6	Testing and Evaluation.....	75
6.1	Manual Testing	75
6.1.1	System testing	75
6.1.2	Unit Testing.....	75
6.1.3	Functional Testing.....	77
6.1.4	Integration Testing.....	77
6.2	Automated Testing:.....	78
7	Conclusion and Future Work	79
7.1	Conclusion	79
7.2	Future Work	79
8	References.....	80

List of Figures

Figure 3. 1 Use Case of E-Garments “Forever”	18
Figure 4. 1 Architecture Diagram of E-Garments “Forever”.....	41
Figure 4. 2 ER Diagram of E-Garments “Forever”	47
Figure 4. 3 Process flow of E-Garments “Forever”.....	53
Figure 4. 4 Sequence Diagram of E-Garments “Forever”	57
Figure 4. 5 Class Diagram of E-Garments “Forever”.....	57
Figure 5. 1 Login Page of E-Garments “Forever”	75
Figure 5. 2 Home Page of E-Garments “Forever”	75
Figure 5. 3 Products Page of E-Garments “Forever”	61
Figure 5. 4 Admin Page of E-Garments “Forever”	77
Figure 5. 5 Cart Page of E-Garments “Forever”	77
Figure 5. 6 Stock Management Page of E-Garments “Forever”	78
Figure 5. 7 Delivery Detail Page of E-Garments “Forever”	79

List of Tables

Table 3. 1 Feature Category for use case	23
Table 3. 2 Login use case.....	26
Table 3. 3 Register use case	27
Table 3. 4 Browse Products use case	28
Table 3. 5 Place Order use case	29
Table 3. 6 Enter Delivery Details use case	30
Table 3. 7 Filter Product use case	30
Table 3. 8 Leave Review use case	31
Table 3. 9 Track Order use case.....	32
Table 3. 10 View Orders use case.....	33
Table 3. 11 View Reviews use case.....	34
Table 3. 12 Update Profile use case.....	35
Table 3. 13 Direct Messaging use case	36
Table 3. 14 AI Assistant Chatbot use case.....	37
Table 3. 15 Product Availability use case.....	38
Table 3. 16 Add Product use case	38
Table 3. 17 Update Product use case	39
Table 3. 18 Delete Product use case	39
Table 3. 19 Make Payments use case.....	40
Table 4. 1 Benefits of Architecture Diagram	45
Table 4. 2 Relationships Summary	51
Table 4. 3 Class Diagram Relationship.....	66
Table 6. 1 Unit Test 1	76
Table 6. 2 Unit Test 2	76
Table 6. 3 Functional Test.....	77
Table 6. 4 Integration Test	78
Table 6. 5 Tool Used.....	78

Chapter 1

Introduction

1 Introduction

The e-commerce industry has experienced rapid growth over the past decade, transforming the way people shop for clothing and fashion accessories. With increasing digitalization and shifting consumer preferences, online shopping has become a preferred choice due to convenience, variety, and accessibility. Despite the rise of numerous e-commerce platforms, many still fail to provide a unified and smooth shopping experience for both customers and store owners. Challenges continue to exist in areas such as responsive website design, efficient product management, secure order handling, and seamless communication between buyers and sellers.

E-Garments Forever is a full-stack based solution developed to address these challenges by offering a smooth, scalable, and user-friendly online shopping ecosystem. Built using modern technologies including MongoDB, Express.js, React.js, and Node.js E-Garments Forever bridges the gap between customers and fashion retailers, ensuring a streamlined and efficient digital purchasing experience.

The core idea behind E-Garments Forever is to provide a flexible, easy-to-use online clothing platform where customers can browse categories, view product details, manage their cart, and place secure orders with ease. The backend is designed with a strong focus on scalability, ensuring that large collections of products, customer profiles, and order information can be stored and managed efficiently using MongoDB. Meanwhile, the front-end UI is designed to be clean, responsive, and user-friendly, enabling smooth navigation across all devices.

This project follows modern development principles, prioritizing modularity, maintainability, and user-centric design. The platform is built to support future growth, allowing for the easy integration of features such as online payment gateways, chatbots, AI-powered product recommendations, and delivery tracking systems as the system evolves.

The subsequent sections of this report will provide a detailed breakdown of the project's background, relevance to course modules, literature review, methodology, system design, and the rationale for selecting the MERN stack. Through this comprehensive study, the report aims to present a clear understanding of the project's architecture, objectives, and alignment with current industry standards and academic requirements.

.

1.1 Brief

The digital revolution has significantly transformed the retail and fashion industry, leading to the emergence of online shopping platforms that cater to the evolving needs of consumers and businesses. E-Garments Forever is a comprehensive, full-stack, web-based e-commerce ecosystem designed to streamline the interactions between customers and clothing retailers. By leveraging modern technologies such as Node.js, Express.js, MongoDB, and React.js, E-Garments Forever offers an integrated solution that enhances user experience, operational efficiency, and scalability.

The platform is architected to support two primary user roles: Customers and Admin/Retailers. Each role is provided with dedicated features tailored to their specific needs and workflows.

Customer Interfaces

- **Web Application**

Built with React.js, the web interface offers similar functionalities, ensuring accessibility across various devices and platforms.

Garments Interface

At the core of E-Garments Forever lies a robust backend system powered by Node.js and Express.js, with MongoDB serving as the primary database. This backend infrastructure handles API routing, authentication, business logic, and data management, ensuring seamless communication between the frontend applications and the database. The system employs RESTful APIs and follows a modular architecture, facilitating maintainability and future scalability.

Key features of the E-Garments Forever platform include

- **User Authentication and Authorization**

Secure login and role-based access control mechanisms to ensure data privacy and integrity.

- **Product Management**

Dynamic product listing creation and editing capabilities for clothing retailers.

- **Order Processing**

Real-time order placement and status updates to enhance customer satisfaction.

- **Review System**

Feedback and rating functionalities to foster trust and continuous improvement.

- **Responsive Design**

User-friendly interfaces optimized for both mobile and web platforms.

The development of E-Garments Forever adheres to modern software engineering principles, emphasizing modularity, scalability, and user-centric design. By integrating cutting-edge technologies and best practices, the platform aims to provide a reliable and efficient solution for the contemporary e-shopping service industry.

1.2 Relevance to Course Modules

The development of E-Garments Forever directly applies knowledge and skills acquired in various course modules typically found in software engineering and computer science curricula. The project not only serves as a practical application of theoretical concepts but also integrates multiple areas of study, enhancing the understanding of full-stack development, mobile application development, database systems, and software engineering methodologies. Below are some key course modules that are highly relevant to this project.

- **Web Technologies**

E-Garments Forever encompasses the entire web development lifecycle, from backend architecture to frontend user interfaces. The backend is powered by Node.js and Express.js, following the principles of RESTful API design and middleware management. On the frontend, the project utilizes React.js, implementing responsive UIs for both mobile and web applications. The seamless interaction between these layers mirrors the content and skills taught in full-stack web development courses, which often focus on creating scalable, API-driven systems.

- **Database Systems**

E-Garments Forever uses MongoDB, a NoSQL database, to store data related to users, orders, garments, and reviews. This system demonstrates an understanding of document-based databases, schema design, and the implementation of CRUD operations. It reflects concepts taught in database systems courses, including data modeling, query optimization, and the benefits of using NoSQL databases for flexible, scalable applications.

- **Software Engineering Methodologies**

The project employs Incremental methodology, often covered in software engineering courses, particularly those focused on project management and development life cycles. The incremental approach to development, combined with version control via GitHub, supports

continuous integration and testing, reflecting industry practices. The project also utilizes modular design and separation of concerns, ensuring that each feature or module of the application can evolve independently while maintaining overall system coherence.

- **Human-Computer Interaction (HCI)**

Human-Computer Interaction (HCI) principles are applied in the design of the user interfaces for both customers and garment owners. The project focuses on creating intuitive, responsive, and aesthetically pleasing UI components that prioritize user experience (UX). The importance of making the system accessible and easy to use, as well as gathering feedback from real users, is a key aspect taught in HCI courses and integrated throughout the development of E-Garments Forever.

- **System Design and Architecture**

The platform architecture of E-Garments Forever adheres to modular design principles, ensuring clear separation between the different components of the system (i.e., backend, web portal). This follows core architectural concepts taught in system integration and architecture modules, such as API-driven development, communication between distributed systems, and scalable software design.

- **Software Project Management**

The development of E-Garments Forever also involved strong elements of project management, including defining milestones, task prioritization, and documentation. The use of a clear, structured README file and regular commits aligns with best practices taught in project management and version control courses.

Through this project, the E-Garments Forever team has been able to integrate knowledge from multiple academic disciplines, reflecting the comprehensive nature of modern software development. The ability to tackle both backend and frontend challenges, while ensuring a focus on user experience, highlights the multidimensional learning that occurs through such an endeavor.

1.3 Project Background

The E-Garments Forever project was conceived as a solution to the growing demand for streamlined and efficient online shopping platforms in the fashion and retail industry. With the rise of digital clothing stores, consumers increasingly expect a seamless and personalized experience

when browsing and purchasing apparel from their preferred brands. Simultaneously, retailers require effective tools to manage products, inventory, customer interactions, and sales data. However, existing solutions often fail to address these needs in a customizable and user-friendly manner.

1.3.1 Problem Statement

Existing online shopping platforms for clothing and fashion often have limitations, including lack of integration with retailer-specific systems, poor management of product listings, and insufficient tools for store owners to manage inventory, product details, and customer interactions. From the customer's perspective, many websites offer inconsistent user interfaces and limited flexibility in browsing and purchasing products. Moreover, from a backend perspective, many platforms struggle with scalability, maintenance, and seamless communication between the various components (e.g., users, products, and orders).

The E-Garments Forever project was thus initiated to create a more flexible, scalable, and user-centric e-commerce platform. Its primary goal was to design a comprehensive system that could address the pain points faced by both customers and retailers, offering them a seamless experience from product selection to order completion.

1.3.2 Target Audience

E-Garments Forever is designed to cater to two main user groups

- **Customers**

Individuals who wish to shop for clothing and fashion accessories via an intuitive web platform. Customers can browse product categories, view detailed product descriptions, add items to their cart, place orders securely, and track the status of their purchases.

- **Garments Owners**

Store owners and managers who wish to manage their online store, update product listings, monitor inventory, track orders, and view sales analytics. The system also provides insights to help retailers optimize their operations and inventory management.

In addition to the customer and Owner-facing interfaces, E-Garments Forever provides an administrative dashboard for overseeing the platform's overall operations..

1.3.3 Project Objectives

The main objectives of the E-Garments Forever project include

- **Streamlining the Ordering Process**

A seamless, user-friendly ordering experience for customers with features like easy navigation, personalized product, and feedback system.

- **Empowering Garments Owners**

Providing Garment owners with the tools they need to efficiently manage their operations, including real-time order status tracking, product management, and sales information.

- **Scalable Architecture**

Building a scalable backend using Node.js, Express.js, and MongoDB to handle growing amounts of data and users without compromising performance or reliability.

1.3.4 Technological Choices

To meet the project's objectives, the following technologies were selected:

- **Node.js**

Used for building the backend APIs due to its scalability and high performance for handling concurrent connections.

- **Express.js**

A lightweight framework that simplifies routing and middleware management for the backend.

- **MongoDB**

A NoSQL database that is highly flexible, making it suitable for handling dynamic data structures like product listings, customer orders, and user profiles.

- **React.js**

Used for building the customer-facing website, offering a responsive and interactive user experience.

1.3.5 System Features

The E-Garments Forever system incorporates various features to enhance both the customer and retailer experience:

- **Customer Features**

Order history, product browsing, reviews, order status tracking, shopping cart management, and profile management.

- **Garment Owner Features**

Product listing creation, inventory management, profile management, sales analytics, order status tracking, and real-time updates.

1.3.6 Challenges and Innovations

Developing a platform like E-Garments Forever required overcoming several technical and operational challenges

- **Real-Time Order Processing**

Ensuring that orders placed by customers are immediately visible to garment owners for timely processing and shipment..

- **Scalability**

Designing the backend to scale as more users and garment owners are onboarded without compromising performance.

- **Cross-Platform Compatibility**

Ensuring that the website works seamlessly across a wide range of devices, screen sizes, and browsers.

The project also involved innovative solutions for creating dynamic product listings, managing inventory efficiently, and ensuring high security for both customer and garment data.

1.4 Literature Review

The e-commerce and online fashion retail industry has seen rapid growth over the last decade, driven by changing consumer behaviors, technological advancements, and increasing internet penetration. The success of global platforms such as Amazon, ASOS, and Zara Online has sparked the development of similar systems, resulting in a competitive and evolving market. This literature review explores various aspects of the online fashion retail industry, including the technologies involved, key design considerations, and challenges that have been addressed in prior research and practice. The review also focuses on backend systems, web development, and database design that are central to projects like E-Garments Forever.

1.4.1 Online Fashion Platforms and Market Trends

Online fashion and clothing platforms have gained significant popularity due to the convenience they offer to consumers, enabling them to browse, compare, and purchase clothing and accessories from the comfort of their homes. According to a study by Statista (2022), the global online fashion market is expected to grow significantly, with projections estimating its value to exceed \$750 billion by 2025. This growth is driven by factors such as the rise in smartphone usage, increased internet penetration, and growing consumer demand for convenience and variety in modern lifestyles.

Several studies have examined the challenges and opportunities within this market, including customer satisfaction, user retention, and the impact of personalized shopping experiences. For instance, Smith et al. (2020) highlighted the importance of efficient product recommendation systems and the need for businesses to optimize user engagement and order fulfillment. Additionally, Lee et al. (2021) discussed the role of responsive web platforms in enhancing customer interaction and streamlining the shopping process [2].

1.4.2 for Building E-Commerce Platforms

The development of online fashion retail systems involves several key technologies, including backend systems, web interfaces, and databases. Many e-commerce platforms use modular or microservices architectures to ensure flexibility and scalability in their backend systems, allowing for independent scaling of individual services and easier maintenance.

1.4.3 Backend Development

The backend of E-Garments Forever platforms typically relies on Node.js [3] and Express.js for their ability to handle asynchronous operations efficiently. Research by Mahajan and Gupta (2019) discusses the advantages of using Node.js in high-traffic environments due to its non-blocking I/O architecture, which allows for processing multiple requests concurrently. Additionally, Express.js provides a minimalistic framework for developing APIs, making it an ideal choice for building scalable and maintainable backend services [4].

A key component of the backend is the database, and many E-Fashion systems prefer NoSQL databases like MongoDB due to their flexibility in handling unstructured data. MongoDB allows for dynamic schemas, making it suitable for applications where the data model is subject to change frequently.

1.4.4 Real-Time Order Status Tracking

Order status checking is an important feature in e-commerce platforms, allowing customers to view the current stage of their orders, such as pending, confirmed, packed, or shipped. Unlike full real-time tracking systems that continuously update the user through automatic notifications, the E-Garments Forever project focuses specifically on providing a simple and clear status-checking option. This means that customers can manually check their order status through the application whenever they want, ensuring transparency and improving user convenience without the need for advanced real-time technologies like Web Sockets.

1.4.5 Challenges in Online Fashion Platforms

Despite the widespread growth of online fashion platforms, several challenges remain that developers need to address. One key issue is scalability as platforms grow in size and the number of users increases, the system must handle a large number of concurrent orders and product catalog requests, ensuring that performance does not degrade. Li et al. (2018) discussed strategies for improving the scalability of e-commerce systems through efficient database design, load balancing, and cloud computing solutions.

Another challenge is real-time data synchronization. For instance, when a customer places an order, the system must ensure that the garment owner receives it immediately, and the order status is updated in real-time across all interfaces. Zhao et al. (2020) highlighted the importance of event-driven architecture for handling these requirements, where events are triggered when specific actions occur, such as order placement or payment [2].

1.4.6 Conclusion from Literature Review

The review of existing literature reveals that e-commerce platforms for clothing and fashion require a combination of effective backend technologies and database systems to function efficiently. By using Node.js, Express.js, and MongoDB, projects like E-Garments Forever can leverage modern technologies to ensure scalability, performance, and maintainability. Additionally, the emphasis on real-time updates and responsive web development through frameworks like React.js aligns with current best practices in the online retail industry.

The challenges identified in the literature, including scalability, real-time data synchronization, and user experience optimization, will inform the design and development decisions for E-Garments Forever. This project seeks to address these challenges while offering a robust solution for both customers and garment owners.

1.5 Analysis from Literature Review

The insights drawn from the literature review provide a foundational understanding of the key factors and technologies that contribute to the development of successful e-commerce platforms. By applying these findings, the E-Garments Forever project can navigate common industry challenges while taking advantage of modern technologies to build an efficient, scalable, and user-friendly online shopping system. This analysis ties together the best practices from previous research and aligns them with the objectives and requirements of E-Garments Forever.

1.5.1 Real-Time Order Status Tracking

As highlighted in the literature, real-time order status tracking is pivotal to customer satisfaction in e-commerce platforms. Studies by Wu et al. (2019) and Zhao et al. (2020) emphasize the need for up-to-the-minute status updates for both customers and garment owners. This aligns with one of the primary objectives of E-Garments Forever: to enhance the user experience by providing accurate and timely order updates. [2]

E-Garments Forever plans to implement a straightforward order status-checking feature that allows customers to view the current stage of their orders whenever they want. After placing an order, both the customer and the garment owner can check the progress through clearly defined stages such as pending, confirmed, processing, packed, or shipped. Although the system does not provide live or automatic updates, it ensures that users can easily access accurate information about their order status. This simple and user-friendly approach enhances transparency and keeps customers informed without the need for complex real-time tracking systems.

1.5.2 Scalable Backend Architecture

The challenge of scaling an e-commerce platform as it grows is a recurrent theme in the literature. According to Li et al. (2018), platforms need to be built with scalability in mind, especially as the number of users and retailers increases. E-Garments Forever addresses scalability by leveraging the combination of Node.js, Express.js, and MongoDB to create a flexible, high-performing backend. Node.js is ideal for handling multiple concurrent requests due to its non-blocking I/O model, which is crucial for platforms where numerous users may be placing orders simultaneously. MongoDB's flexible schema model allows E-Garments Forever to scale its database efficiently without rigid constraints. Whether handling customer orders, product catalogs, or user profiles, MongoDB provides a flexible and scalable solution for the diverse types of data the platform manages.

1.5.3 User Experience (UX) and User Interface (UI) Design

UI/UX design plays a significant role in user satisfaction, as pointed out by Norman (2013), emphasizing that intuitive and user-friendly designs are crucial for online platforms. For an e-commerce platform like E-Garments Forever, ensuring ease of navigation and a streamlined checkout process is vital to keeping users engaged and minimizing cart abandonment [5].

To meet these expectations, E-Garments Forever will implement a responsive design that adjusts seamlessly to various screen sizes, ensuring a consistent experience across devices. Additionally, the intuitive design will enable users to easily browse products, add items to their cart, and view

order statuses. Features like order history, product search, and user reviews will be prominently displayed to enhance engagement and make the platform more personalized.

1.5.4 Real-Time Data Synchronization

The need for real-time data synchronization across multiple platforms is another critical element discussed in the literature. As noted by Zhao et al. (2020), e-commerce platforms must ensure that when an order is placed, the data is immediately reflected across all relevant systems. For E-Garments Forever, this means that once a customer places an order, the garment owner should receive the order in real-time, and both the customer and garment owner should be able to view the order's status without delay.

E-Garments Forever plans to use an event-driven approach to maintain smooth synchronization between customers, garment owners, and the admin dashboard. Through event-based updates handled within the system's backend, any change in order status—such as order confirmed, processing, shipped, or delivered will be reflected across all relevant interfaces. This ensures that customers can check their order status accurately whenever they access the platform, and garment owners can manage their workflow efficiently based on the most recent information available [2].

1.5.5 Conclusion

The analysis of the literature review has revealed several key insights that directly inform the development of E-Garments Forever. By leveraging real-time data synchronization, scalable backend technologies, responsive web development, and user-centered design, E-Garments Forever will address critical challenges in the online fashion retail industry while ensuring an optimal experience for both customers and garment owners.

With these lessons in mind, E-Garments Forever is positioned to provide a comprehensive, efficient, and scalable solution for modern e-commerce, making use of the latest technological advancements to meet the demands of today's fast-paced, convenience-driven market.

1.6 Methodology and Software Lifecycle for This Project

The project was developed using the Incremental Software Development Life Cycle (SDLC). This model allowed the team to build the application in successive iterations. Each increment included specific functional modules such as user registration/login, product catalog browsing, order placement, payment processing, and order status tracking. Modules were tested and validated individually before integrating them into the full system. This methodology provided flexibility, early detection of issues, and scope for refinement.

Rationale behind Selected Methodology

The Incremental Model has been chosen as the methodology for the E-Garments Forever project due to its flexibility, iterative approach, and focus on gradual improvements. This model divides the project into smaller, manageable increments, each of which adds a specific set of features to the system. Each increment is developed, tested, and delivered in stages, allowing for progressive refinement and adjustments based on user feedback and changing requirements.

The decision to adopt the Incremental Model is driven by the following key factors:

Flexibility and Adaptability

The Incremental Model provides significant flexibility, making it ideal for projects where requirements may evolve over time, as in the case of E-Garments Forever. By delivering features in increments, the team can adjust to user feedback and market demands after each stage of development. This iterative approach allows for continuous improvements without the need for a complete overhaul of the system, which is particularly beneficial in the fast-paced online retail industry, where user preferences and technological trends can shift quickly.

Early Deliverables and Feedback

With the Incremental Model, parts of the application can be delivered early and tested in real-world scenarios, providing valuable user feedback from both Garment owners and customers. For example, early increments may include basic functionality such as order placement and product browsing, while later increments could focus on advanced features like status tracking. This early feedback helps identify potential issues before they become larger problems, ensuring that the final product aligns closely with user needs and expectations.

For E-Garments Forever, this approach allows the development team to release early versions of the app for customer and garment use, enabling beta testing and gathering real-world data. The feedback obtained from these early stages will be crucial in shaping the design and functionality of later increments, ensuring that the platform continuously meets user demands.

Risk Management

The Incremental Model allows for the identification and mitigation of risks at each stage of development. By focusing on smaller, more manageable sections of the project, potential risks related to features, scalability, or security can be identified early and addressed before they escalate. For instance, if any issues arise with the backend architecture in the initial increments, these problems can be resolved before moving on to more complex functionality, such as real-time tracking or inventory management.

This continuous evaluation and reassessment of risks in each increment minimize the likelihood of major failures or delays at the later stages of the project.

Continuous Improvement

By breaking the project into increments, the development team can focus on delivering value with each iteration, allowing the product to evolve continuously. Each increment can build on the lessons learned from the previous ones, improving both the quality of the features and the overall user experience. The incremental approach supports continuous integration of new features and refinements, allowing for more robust testing and quality assurance before each new release.

For E-Garments Forever, this means that as new features like real-time order status tracking, and review systems are developed, they can be integrated smoothly into the existing platform without significant disruption. This continuous improvement ensures that the platform remains up-to-date with the latest technological trends and user demands.

Clear Milestones and Progress Tracking

The Incremental Model also provides clear milestones, making it easier to track progress. Each increment represents a tangible, deliverable product that can be tested and reviewed, allowing both the development team and stakeholders to assess the status of the project. These milestones provide a clear roadmap, ensuring that E-Garments Forever stays on track and progresses toward the final goal. This clarity helps with project management and ensures that the development team meets deadlines for each increment.

Support for Ongoing Development

Given the nature of the E-Garments Forever project, which is designed to be continuously updated and enhanced based on user needs, the Incremental Model is well-suited for the ongoing maintenance and evolution of the platform. As new technologies emerge or customer preferences change, new increments can be added to the platform without disrupting existing functionality.

Conclusion

The Incremental Model provides the ideal framework for the E-Garments Forever project, offering flexibility, early feedback, risk management, resource optimization, and continuous improvement. By delivering the platform in manageable increments, the development team can ensure that the project is adaptable to changes, scalable, and capable of meeting the needs of both customers and garment owners. This approach aligns with the project's goals of providing a high-quality, user-centric e-commerce platform that evolves in response to user feedback and market demands.

Chapter 2

Problem Definition

2 Problem Definition

The E-Garments Forever project aims to address the challenges faced by the online fashion retail industry, specifically targeting the complexities associated with e-commerce platforms for garment owners and customers. The online fashion market is experiencing rapid growth, with increasing demands for efficient, user-friendly solutions that connect consumers with clothing and accessory retailers. However, many existing platforms lack seamless integration, intuitive user interfaces, and efficient backend systems that ensure smooth order processing, customer satisfaction, and inventory management.

The E-Garments Forever platform is designed to provide a comprehensive solution to these challenges by offering a multi-faceted application suite that includes a customer-facing web app, a garment owner dashboard, and backend services, all built to facilitate the process of browsing, ordering, and managing clothing products in a more efficient, accessible, and scalable manner. However, the development of such a platform brings about its own set of technical, operational, and user-experience challenges that need to be addressed.

These challenges are reflected in several key areas

- **User Experience (UX) Issues**

The need for an intuitive, responsive, and user-friendly interface for both customers and garment owners.

- **Seamless Integration**

The difficulty in integrating various modules (frontend apps, backend services, and databases) to ensure smooth communication and data flow between users and garments.

- **Scalability**

Building a scalable infrastructure that can handle increasing traffic and growing user and garment bases as the platform evolves.

- **Performance and Efficiency**

Ensuring fast and efficient performance in all areas of the app, especially for time-sensitive transactions like placing and processing orders.

- **Real-Time Processing**

Providing a system for real-time updates, including order status stock updates, and Product changes.

These challenges will be tackled using a modular and iterative development approach (the Incremental Model) that allows for the gradual addition of features while continuously improving upon existing ones. By developing the platform incrementally, feedback can be integrated early, ensuring that the final product meets user expectations and addresses the core challenges in the online fashion retail domain.

2.1 Problem Statement

In the evolving landscape of online retail, fashion e-commerce has emerged as a critical area where efficiency, convenience, and real-time responsiveness are paramount. While numerous platforms exist that attempt to connect customers with garment retailers, most suffer from over-complexity, limited customization for garment owners, poor integration between customer interfaces and retailer operations, or technical limitations in terms of scalability and performance.

The E-Garments Forever project is designed to address these limitations by building a modular, full-stack solution that facilitates seamless online shopping, garment management, and customer interaction through dedicated web applications and dashboards supported by a robust backend infrastructure. The current market lacks a unified, extensible, and lightweight system that provides the following in a cohesive environment:

- A web for customers that offers real-time browsing of clothing catalogs, smooth product selection, cart management, and order tracking with minimal latency.
- A dashboard for garment owners that allows them to manage product listings, view and respond to customer orders in real time, and update stock availability dynamically.
- A centralized backend system using Node.js and Express.js connected to MongoDB that supports secure data flow, user authentication, media uploads, order management, and real-time database interaction for multiple actors (customers and garment owners).

Despite the availability of partial solutions in the market, there remains a lack of customizable, educationally-driven platforms that combine technical depth with practical usability. Educationally, this project serves as an advanced exploration of full-stack development, integrating backend API design, database modeling, state management, asynchronous processing, and modern UI/UX practices.

Thus, the core problem addressed in this project is:

"How can a full-featured, scalable, and user-friendly online fashion retail system be designed and implemented across web platforms to ensure seamless interaction between customers and garment owners, supported by a robust backend capable of managing real-time operations?"

This project sets out to solve this problem by developing a complete system that not only meets technical requirements but also aligns with user expectations and business processes of modern online fashion retail.

2.2 Deliverables and Development Requirements

The development of the E-Garments Forever system is structured around a clear set of deliverables, each targeting a distinct functional area of the platform. These deliverables are aligned with the project's objectives to create a scalable, modular, and real-time e-commerce system that integrates customer and garment owner interactions via web and mobile interfaces, all powered by a unified backend infrastructure.

2.2.1 Key Deliverables

Backend System (Node.js, Express.js, MongoDB)

- RESTful API architecture with secure route handling.
- User and garment authentication (JWT-based).
- CRUD operations for products, garments profiles, user data, and orders.
- Real-time order status updates.
- Image upload and management system integrated with cloud/local storage.
- Modular controller, route, and model structure to support scalability and maintainability.
- Integration of the Incremental Software Development Model for progressive feature rollouts.

User Web Portal (React.js)

- Responsive web interface for browsing and purchasing garments.
- Feature parity with mobile app: add to cart, checkout, reviews, and wishlist.
- Adaptive design for various screen sizes and user preferences.

2.2.2 Development Requirements

To implement the above deliverables, the following technical, and environmental requirements must be met:

Technical Requirements

- Node.js and Express.js as the primary backend framework.
- MongoDB for document-oriented database storage.
- React.js for the web-based interface.
- Postman for API testing [6].
- Git and GitHub for version control and collaboration.
- UUID for image and record uniqueness.
- JWT for secure authentication.

Environmental Requirements

- Development environment: VS Code, Expo Go, Android Emulator/Physical device.
- Testing environment: Expo Metro Bundler or development server deployment.
- MongoDB Atlas or locally hosted MongoDB instance with replicas setup.

2.3 Current System

Prior to the development of E-Garments Forever, there were very few existing centralized systems that offered a unified solution for managing online garment sales across both mobile and web platforms from the perspective of customers and garment owners.

2.3.1 Lack of Integration

Most third-party e-commerce platforms are not integrated directly with garment store workflows. Store owners have minimal control over their product listings, stock updates, and customer data. Additionally, store owners often rely on external marketplaces while still managing order processing manually.

2.3.2 User Experience Challenges

Customers often face inconsistent experiences depending on the platform being used. Web portals may lack real-time inventory updates or responsiveness, and often fail to match modern UI/UX standards or synchronize with other interfaces.

2.3.3 Manual Inventory and Product Management

Store staff frequently update product listings and stock manually, leading to inconsistencies between available items and what customers see on the platform. This results in customer dissatisfaction, canceled orders, and lost sales opportunities.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Derived from the SRS, this chapter describes how the system's needs were gathered, documented, and represented.

3.1 Use Cases Diagram(s)

The use case diagram of E-Garments Forever (Figure 3.1) shows interactions between Customers, Admins, and the Payment Gateway. Customers perform actions like registration, ordering, tracking, and Apply Coupon, while Admins manage products, orders, and reviews [7].

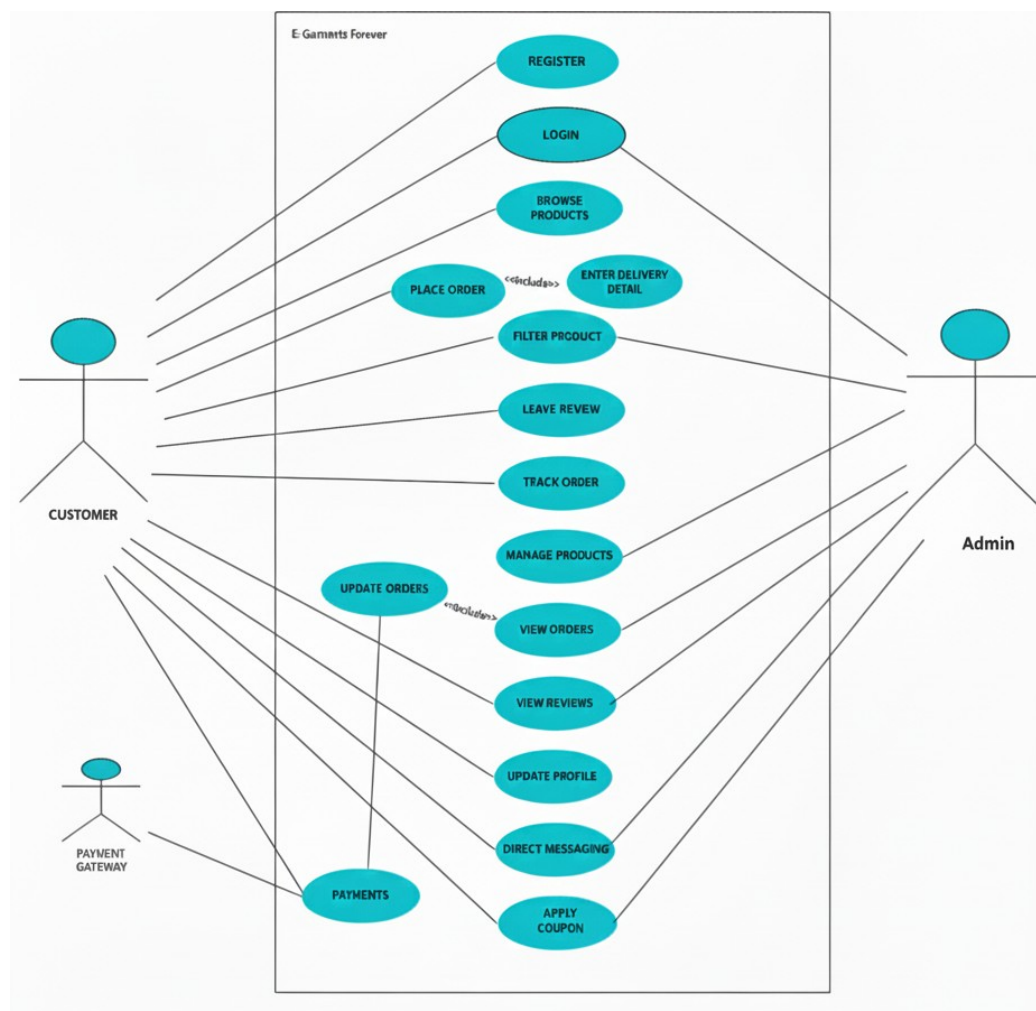


Figure 3. 1 Use Case of E-Garments "Forever"

1. Overview of the Use Case Diagram

The use case diagram E-Garments Forever (Figure 3.1) represents the functional model of the online shopping platform. It visually shows how three main actors Customer, Admin, and Payment Gateway interact with the system.

Each oval in the diagram represents a use case, meaning a function the system provides. Lines between actors and use cases represent how users initiate actions inside the system.

This diagram helps in understanding how different users perform tasks like browsing products, placing orders, managing products, viewing reviews, and processing payments.

2. Primary Actors in the System

There are three main actors identified:

- **Customer**

End-users who shop for garments online. They perform actions like registering, logging in, browsing products, placing orders, tracking orders, reviewing product.

- **Admin**

Represents the garment store owner or platform manager. They manage products, update and view orders, handle customer reviews, and oversee profile and messaging features.

- **Payment Gateway**

A third-party system responsible for securely handling customer payments.

Each actor has specific goals and actions they want to perform using the system.

3. Use Cases for the Customer

The **Customer** actor is connected with the following use cases:

- a) **Register**

Customers must first create an account in the system. The **Register** use case allows users to input personal information such as name, email, phone number, and password to sign up.

- b) **Login**

After registration, customers use the **Login** function to access their accounts. This typically includes authentication via email and password.

c) Browse Product

Customers can explore all garment items available in the store. This includes viewing product images, descriptions, categories, and prices.

d) Filter Products

Customers can narrow down product listings by category, price range, size, or other attributes.

e) Place Order

Customers select products, add them to the cart, and confirm an order.

f) Enter Delivery Details

A required step when placing an order. Customers provide:

- Address
- Contact info
- Delivery notes

g) Leave Review

After receiving products, customers can leave star ratings and text reviews for purchased items.

h) Track Order

Allows customers to check their order progress such as:

- Confirmed
- Packed
- Shipped
- Delivered

i) Update Profile

Customers can update their personal details, password, or preferences.

j) Direct Messaging

Customers can directly communicate with support or admin through WhatsApp.

k) Apply Coupon

Customers can enter a coupon code to receive a discount on their order.

4. Use Cases for the Admin

The **Admin** actor interacts with the system for backend operations, focusing on managing orders and products. The following use cases are associated

a) Manage Products

The Admin can add, edit, or delete garments. This includes updating stock, images, prices, and descriptions.

b) View Orders

The Admin can see customer orders, including order details, payment status, and delivery information.

c) Update Orders

Admins update order progress confirming, packing, shipping, or completing it. These updates reflect in the customer's Track Order section.

d) View Reviews

Admins read reviews left by customers to understand product quality and customer satisfaction.

5. Use Cases Involving the Payment Gateway

Payments

The Payment Gateway interacts with the system to securely process customer payments. This includes:

- Verifying payment details
- Confirming successful transactions
- Sending payment status updates to the system

This ensures the Admin and Customer receive accurate payment information.

6. Shared Use Cases Between Customer and Admin

Some actions, although initiated by the customer, also require or involve the admin, particularly for communication and process flow.

a) Track Order

While this appears as a customer activity, it also relies on the garment updating order statuses.

b) Place Order

Though initiated by the customer, the garment plays a role in receiving and preparing the order.

c) View Reviews

The customer leaves reviews, and the garments reads them. Thus, this is a shared use case.

d) Direct Messaging

Customer sends messages through WhatsApp to Admin views and replies.

7. Relationships and Connectors

- **Solid lines** indicate that the actor is directly involved in the use case.
- **Dashed arrows with <<include>>** represent a mandatory sub-process. For example, “Enter Delivery Details” is an included use case within “Place Order,” meaning it is required for completion.

8. System Boundary

The large rectangle titled **E-Garments “Forever”** represents the system boundary. All use cases inside this rectangle are part of the system. Anything outside (e.g., the actors) interacts with the system but is not part of it.

9. Functional Coverage of the System

This use case diagram comprehensively covers the functionality of a online shopping system:

Table 3.1 presents the feature categories of the E-Garments Forever platform. It includes user authentication for registration and login, product navigation to explore items and check availability, and order processing for adding products to the cart, placing orders, and tracking deliveries. Additionally, it covers admin operations, feedback mechanisms, coupon application, and secure payment integration.

Table 3. 1 Feature Category for use case

Feature Category	Description
User Authentication	Customers register and log in to the system.
Product Navigation	Users explore clothing categories, view product details, and check availability.
Order Processing	Users add products to the cart, place orders, choose delivery details, and track their order status.
Admin Operations	Admin manages products, updates inventory, handles orders, and views customer activity.
Feedback Mechanism	Customers submit reviews and ratings for products; admin can monitor feedback.
Apply Coupon	Users can enter a coupon code to receive a discount on their purchase.
Payment Integration	Secure online payments are processed through the integrated payment gateway.

10. Benefits of This Use Case Diagram

- **Clarity of Responsibilities**

It clearly shows what each actor can and cannot do.

- **Planning and Development**

Useful during system design to define scope and allocate development tasks.

- **Testing and Validation**

Serves as a basis for creating test cases to ensure system functionality.

11. Potential Enhancements

The diagram can be improved by:

- Adding **Admin** actor for platform management.
- Including **Payment Processing** use case.
- Representing **Customer Support** features like chatting or raising complaints.

3.2 Detailed Use Case

The tables below indicate a comprehensive use case template filled out for the all use cases in the Forever system.

Table 3.2 Login use case

Use case id	01
Use case name	Login
Actors	- Customer - Admin
Description	A user logs into their Forever account
Trigger	User clicks "Login" button
Pre-condition	The user has a registered account
Postcondition	The user is logged into their account dashboard
Normal flow	- User enters email/phone and password - System validates credentials - If valid, system logs user into their account
Alternative flow	- Invalid credentials - Error message displayed to re-enter details
Exceptions	- Incorrect email/password - Network error
Business rules	System requires internet connectivity to authenticate login
Assumptions	- Users must log in to place orders or manage a store. - A "Forgot Password" option will be available.

Table 3.3 Register use case

Use case id	02
Use case name	Register
Actors	Customer
Description	A non-registered user creates a new account
Trigger	Customer clicks "Register" button

Pre-condition	Customer does not have an existing account
Postcondition	Customer's user profile is created
Normal flow	- System displays registration form - Customer enters required details and submits - System validates details and creates user profile - Customer is registered and can now login
Alternative flow	- Cancel Registration - Customer clicks cancel, registration stops - Invalid Data - System alerts customer to correct errors
Exceptions	- Existing email - "Email already registered" message - Incomplete form - "Please fill all fields" message
Business rules	Active internet connection required
Assumptions	- All users (customers & retailers) must register before accessing features. - Email and phone verification will be required.

Table 3.4 Browse Products use case

Use case id	03
Use case name	Browse Products
Actors	Customer
Description	Customers can browse Garments products to view product details
Trigger	Customer clicks "Products" button on a garments page
Pre-condition	Customer is logged into their account
Postcondition	Customer sees the product for the selected garments
Normal flow	- Customer selects "Products" option - System displays that garments products - Customer views products names, photos, prices

Alternative flow	- Garments has no products - Support notification generated
Exceptions	- Server error - Generic error message
Business rules	Garments must have an active products uploaded
Assumptions	- Customers can browse products without logging in. - Search, filters, and categories will enhance browsing.

Table 3.5 Place Order use case

Use case id	04
Use case name	Place Order
Actors	Customer
Description	Customers can place orders for product from a garments through this application.
Trigger	Customer selects desired garments products and proceeds to checkout.
Precondition	Customer is logged into their account and has selected some garments products.
Postcondition	Garments receives the order request and confirms it.
Normal flow	- Customer selects desired products. - Customer browses through the products. - Customer adds desired products to the cart. - Customer proceeds to checkout. - Customer confirms order details. - System sends order request to garments. - Garments receives and confirms the order. - Customer receives confirmation of order.
Alternative flow	- products is out of stock - System notifies the customer - Customer remove products or select alternative - Garments is closed

	- System notifies the customer
Exceptions	- Server error - Generic error message
Business rules	Customer must be logged in to place an order. Selected products must be available for delivery. Products added to the cart must be available and in stock.
Assumptions	- Customers must be logged in to order. - Stock availability will be checked before confirming an order.

Table 3.6 Enter Delivery Details use case

Use case id	05
Use case name	Enter Delivery Details
Actors	Customer
Description	Customer provides details for their delivery address and options
Trigger	Customer clicks "Delivery Info" on place order screen
Precondition	Customer account is logged in
Postcondition	Delivery details are captured
Normal flow	- Customer enters name, address, phone - Customer clicks confirm
Alternative flow	- Invalid data - Errors shown to correct
Exceptions	- Missing fields - "Please fill all required fields" - Server error - Generic error message
Business rules	Address must be within delivery zone
Assumptions	- Customers must provide a valid delivery address. - The system will allow pre-saved addresses for faster checkout.

Table 3.7 Filter Product use case

Use case id	06
Use case name	Filter Product
Actors	- Customer - Admin
Description	Allows customers to filter products based on different criteria such as category, price range, size, brand, color, and discount offers.
Trigger	The customer selects a filtering option while browsing products
Precondition	The user is on the product browsing page.
Postcondition	The system displays products that match the selected filter criteria.
Normal flow	- Customer navigates to the product listing page. - Customer selects one or multiple filter options (e.g., price, size, color). - System applies the filters and updates the product list accordingly. - Customer reviews the filtered results. - Customer can reset or modify filters if needed.
Alternative flow	- If no products match the selected criteria, the system displays a "No products found" message and suggests similar products. - Errors shown to correct
Exceptions	- System fails to load filtered results due to a server error. - "Please fill all required fields" - Server error - Generic error message
Business rules	- Filters should be dynamic and based on available product attributes. - Users can apply multiple filters at once. - Reset option should be available to remove all applied filters.
Assumptions	- Customers can filter products by price, size, color, brand, and discounts. - Multiple filters can be applied simultaneously.

Table 3.8 Leave Review use case

Use case id	07
Use case name	Leave Review
Actors	Customer
Description	Customers can leave reviews and ratings for products they have ordered through this application.
Trigger	Customer selects a products they have ordered and chooses to leave a review.
Pre-condition	Customer is logged into their account and has previously ordered the garments products.
Postcondition	Review is submitted for the selected products.
Normal flow	- Customer navigates to the "Recent Orders" section. - Customer selects the specific order containing the garments products they want to review. - Customer finds the desired garments products within the order details. - Customer selects the option to leave a review for the chosen the products. - Customer writes a review for the products. - Customer submits the review and rating. - System records the review and rating for the respective products. - Other users can view the review and rating for the garments products.
Alternative flow	- Customer decides to not leave a review - Customer exits the review submission process.
Exceptions	- Server error - Generic error message returned
Business rules	- Customer must be logged in to leave a review. - Customer can only leave a review for products they have previously ordered. - Customers cannot modify or delete their reviews once submitted
Assumptions	- Only customers who have purchased a product can leave a review. - Reviews will be moderated to prevent spam or inappropriate content.

Table 3.9 Track Order use case

Use case id	08
Use case name	Track Order
Actors	Customer
Description	Customers can track the status and location of their order from the moment it is placed until it is delivered.
Trigger	Customer selects the option to track their active order.
Precondition	Customer is logged into their account and has an active order.
Postcondition	Customer can monitor the status and location of their order in real-time.
Normal flow	- Customer selects the active order from their account dashboard. - System displays the current status of the order (e.g., confirmed, preparing, on the way, delivered). - System provides estimated delivery time of the order. - Customer receives notifications for important order status updates (e.g., order confirmed, order out for delivery). - Once the order is delivered, the system updates the status to "delivered" and provides a summary of the completed order.
Alternative flow	- Delivery driver encounters a delay - Notify the customer of delay
Exceptions	- Server error - Generic error message
Business rules	- Customer must be logged in to track an order. - Order tracking information should be updated in real-time.
Assumptions	- Customers can track their order status in real time. - Notifications will be sent for major updates (Processing, Shipped, Delivered).

Table 3.10 View Orders use case

Use case id	9
Use case name	View Orders
Actors	- Customer - Admin

Description	Admin can view the list of orders received from customers through this application.
Trigger	Admin accesses the "View Orders" section within their Garments account.
Pre-condition	Admin is logged into their Garments account.
Postcondition	Admin can view details of each order received, including order contents, customer details, and order status.
Normal flow	- Admin navigates to the "View Orders" section within their garment account dashboard. - System displays a list of all received orders. - Admin selects a specific order from the list to view its details. - System displays the details of the selected order. - Current status of the order (e.g., confirmed, preparing, on the way, delivered) - Admin can update the status of the order (e.g., mark as prepared, mark as delivered) based on the progress. - System automatically updates the order status and notifies the customer accordingly.
Alternative flow	- No orders received - System displays a message indicating that there are no orders currently.
Exceptions	- Server error - Generic error message
Business rules	- Only Admin with appropriate permissions can access the "View Orders" section. - Order details should be displayed accurately and updated in real-time to reflect the latest status. - Admin should handle orders promptly and update their status to keep customers informed about the progress of their orders.
Assumptions	- Customers and retailers can view order history. - Order details (products, price, status, expected delivery) will be shown.

Table 3.11 View Reviews use case

Use case id	10
Use case name	View Reviews
Actors	- Customer - Admin
Description	Admin can check the reviews and ratings left by users on products offered by their garments through the application.
Trigger	Admin accesses the "User Reviews" section within their garments account.
Precondition	Admin is logged into their Garments account.
Postcondition	Admin can view and respond to user reviews.
Normal flow	- Customer navigates to "Profile Settings." - The system displays the current profile details. - Customer edits the necessary fields. - Customer saves the changes. - System updates the information and confirms the update.
Alternative flow	- If the customer does not provide valid information, the system displays an error message.
Exceptions	- Server error - Generic error message
Business rules	- Only Admin with appropriate permissions can access the "User Reviews" section. - Reviews should be displayed accurately and updated in real-time to reflect the latest feedback from users.
Assumptions	- Customers can see reviews left by other users. - Retailers can view and respond to reviews about their products.

Table 3.12 Update Profile use case

Use case id	11
Use case name	Update Profile
Actors	- Customer

Description	Allows customers to update their personal details like name, contact information, address, and preferences within their account.
Trigger	The customer selects the "Update Profile" option.
Precondition	- The user is logged into their account.
Postcondition	- The updated details are saved and reflected in the user's profile.
Normal flow	- Customer navigates to the profile section. - Customer selects "Edit Profile." - Customer updates personal details (name, email, phone, address). - System validates the inputs and saves the changes. - Updated details are displayed in the user's profile.
Alternative flow	- Invalid input detected (e.g., wrong email format, phone number). - System prompts user to correct the data before saving.
Exceptions	- Server error - Generic error message
Business rules	- Customers can only update their own profiles. - System validates email and phone number formats before saving.
Assumptions	- Users (customers & retailers) can update name, email, phone, and profile picture. - Security checks will be required for sensitive changes (e.g., password updates).

Table 3.13 Direct Messaging use case

Use case id	12
Use case name	Direct Messaging
Actors	- Customer - Garments
Description	Customers and sellers can communicate directly through the platform for inquiries about products, orders, and delivery details.
Trigger	A user selects the messaging option to start a conversation.
Precondition	Both parties must be logged in to the platform.
Postcondition	Messages are sent and received in real-time.

Normal flow	- User selects a contact to message. - User types and sends a message. - The recipient receives the message instantly. - System allows users to continue chatting.
Alternative flow	- If the recipient is offline, the system stores messages for later retrieval.
Exceptions	- Server error - Message sending failure due to connectivity issues
Business rules	- Users can only send messages to relevant contacts (customers to sellers, sellers to customers). - System stores message history for future reference.
Assumptions	- Customers and retailers can communicate regarding orders. - Chat history will be stored temporarily for support purposes.

Table 3.14 Apply Coupon use case

Use case id	13
Use case name	Apply Coupon
Actors	- Customer - Admin
Description	Allows customers to enter a coupon code to receive a discount on their purchase. Admins can create, update, or manage coupon codes in the system.
Trigger	- Customer selects or enters a coupon code during checkout. - Admin adds or updates a coupon in the system.
Precondition	- Customer is logged in and has items in the cart. - Admin is logged in to the admin dashboard.
Postcondition	Discount is applied to the customer's order (if valid). Coupon details are saved or updated in the system.
Normal flow	- Customer opens the checkout page. - Customer selects the "Apply Coupon" option. - Customer enters a valid coupon code.

	- System validates the coupon code. - If valid, the discount is applied, and the order summary is updated. - Customer proceeds to complete the payment.
Alternative flow	If the coupon is invalid or expired, the system displays an error message for the customer. If the admin enters invalid data, the system shows an error and requests correction.
Exceptions	Coupon validation fails due to server issues. Coupon cannot be applied to restricted items.
Business rules	- Only one coupon can be applied per order (unless specified). - Coupons may have minimum order value or expiry date restrictions. - Admins are responsible for coupon creation and validity.
Assumptions	- The system maintains a database of valid coupons. - Discounts are calculated accurately and reflected in real time.

Table 3.15 Product Availability use case

Use case id	14
Use case name	Product Availability
Actors	- Admin
Description	Admin checks and updates stock levels to ensure availability for customers
Trigger	Admin selects "Product Availability" option.
Precondition	- Admin is logged in.
Postcondition	- Product stock information is updated in the system.
Normal flow	- Admin logs in. - Navigates to Inventory Management. - Checks stock levels. - Updates availability if needed.
Alternative flow	If stock is low, system notifies admin to restock.
Exceptions	- System downtime may prevent updates.

Business rules	- Only authorized admins can modify stock. - Products with zero stock should not
Assumptions	- The system syncs real-time stock levels

Table 3.16 Add Product use case

Use case id	15
Use case name	Add Product
Actors	- Admin
Description	Admin adds new garments by entering details such as name, price, category, description, and images.
Trigger	Admin selects "Add Product" option.
Precondition	Admin has required permissions.
Postcondition	New product is successfully added to the catalog.
Normal flow	- Admin logs in. - Navigates to Product Management - Fills in product details - Submits the new product.
Alternative flow	If images are missing, the system prompts the admin to upload them.
Exceptions	- Incorrect data format prevents submission.
Business rules	- Only admins can add new products. - Each product must have a unique identifier (SKU).
Assumptions	- The system validates product details before adding them.

Table 3.17 Update Product use case

Use case id	16
Use case name	Update Product
Actors	- Admin
Description	Admin modifies existing product details such as price, stock, and descriptions.
Trigger	Admin selects a product for editing.

Precondition	Product exists in the catalog.
Postcondition	Product details are successfully updated.
Normal flow	- Admin logs in. - Navigates to Product List. - Selects a product to edit. - Updates details and saves changes.
Alternative flow	If the price changes drastically, the system asks for confirmation
Exceptions	- System error may prevent saving changes.
Business rules	- Only admins can update product details. - Price changes require admin approval.
Assumptions	The system logs changes for audit purposes.

Table 3.18 Delete Product use case

Use case id	17
Use case name	Delete Product
Actors	- Admin
Description	Admin removes discontinued or out-of-stock products from the catalog.
Trigger	Admin selects "Delete Product" option.
Precondition	Product exists in the system.
Postcondition	Product is removed from the catalog.
Normal flow	- Admin logs in. - Navigates to Product List. - Selects the product for deletion. - Confirms deletion.
Alternative flow	If the product has pending orders, the system prevents deletion.
Exceptions	System downtime may cause failure.
Business rules	- Only authorized admins can delete products. - Deleted products cannot be restored.
Assumptions	The system maintains logs of deleted products.

Table 3.19 Make Payments use case

Use case id	18
Use case name	Make Payments
Actors	- Customer - Payment Gateway
Description	Customers can complete purchases using different payment methods.
Trigger	Customer proceeds to checkout and selects a payment method
Precondition	- The user has added products to the cart and is logged in.
Postcondition	- Payment is processed, and the order is confirmed.
Normal flow	- Customer selects payment method (COD, Stripe etc.). - System redirects to the payment gateway. - Customer enters payment details. - System processes the transaction. - Payment confirmation is displayed, and an order is placed.
Alternative flow	If payment fails, system prompts user to retry or select a different payment method.
Exceptions	- Transaction failure due to bank issues. - Network connection lost during payment.
Business rules	- System must ensure secure transactions. - Orders are not confirmed until payment is verified.
Assumptions	- Multiple payment methods (credit/debit card, mobile wallets, COD) will be supported. - Payments will be securely processed via an integrated gateway.

3.3 Functional Requirements

The project is divided into two main modules:

3.3.1 Customer Module

- **Registration/Login**

Customers can create their accounts and securely log in to access personalized features such as saved profiles, order history, and recommended products.

- **Home Page**
Displays featured clothing products, seasonal collections, discounts, and quick navigation links to categories such as Men, Women, Kids, and Accessories.
- **About Page**
Provides detailed information about E-Garments Forever, including the platform's mission, values, vision, and commitment to quality service.
- **Contact Page**
Offers customer support options including email, phone number, and AI chatbot for immediate assistance.
- **Product Browsing**
Customers can explore various clothing categories, filter products by size, color, price, or brand, and view detailed product descriptions and images.
- **Placing Orders**
Customers can add products to their cart, enter delivery details, choose payment method, and confirm their order using a secure checkout system.
- **Tracking Orders**
Customers can track order status in real time—from order confirmation to packaging, shipping, and final delivery.
- **Reviewing Product**
Customers can leave ratings and written feedback on purchased clothing products to help others make informed buying decisions.

3.3.2 Admin Module

- **Managing Garment Profile**
Garment owners can update store details such as shop name, contact info, working hours, and profile/shop images.
- **Product Management**
Allows garment owners to add new clothing products, update product details, adjust prices, manage stock levels, and remove outdated products.

- **Viewing Orders**

Displays all customer orders with product details, delivery addresses, and order statuses so the owner can process and dispatch orders efficiently.

- **View Reviews**

Garment owners can view customer reviews on their products and respond to improve service quality and customer engagement.

- **Sales insights**

Provides detailed sales analytics such as top-selling products, total revenue, and customer purchase patterns to support business decision-making.

These requirements are outlined to ensure the platform meets all expected functions and delivers a comprehensive ordering experience.

3.4 Non-Functional Requirements

The system also incorporates several key non-functional requirements:

- **Usability**

Clean, user-friendly interface across devices to ensure smooth navigation and ease of use for all users, including first-timers.

- **Performance**

Fast load times and real-time updates are ensured by optimized backend services and lightweight front-end components.

- **Portability**

Works across Android, iOS, and major web browsers using ReactJS for consistent performance across platforms

- **Scalability**

System architecture supports adding more garments and customers without performance loss, using cloud-based infrastructure and modular design.

Chapter 4

Design and Architecture

4 Design and Architecture

From the SDD, this chapter covers the structural and technical design of the system.

4.1 System Architecture

The architecture diagram of E-Garments Forever (Figure 4.1) uses a client–server architecture where the ReactJS frontend displays the interface and temporarily stores user data. The backend, built with Node.js and Express.js, processes business logic and communicates with MongoDB for data storage. External services such as a Geolocation API support features like real-time order tracking.

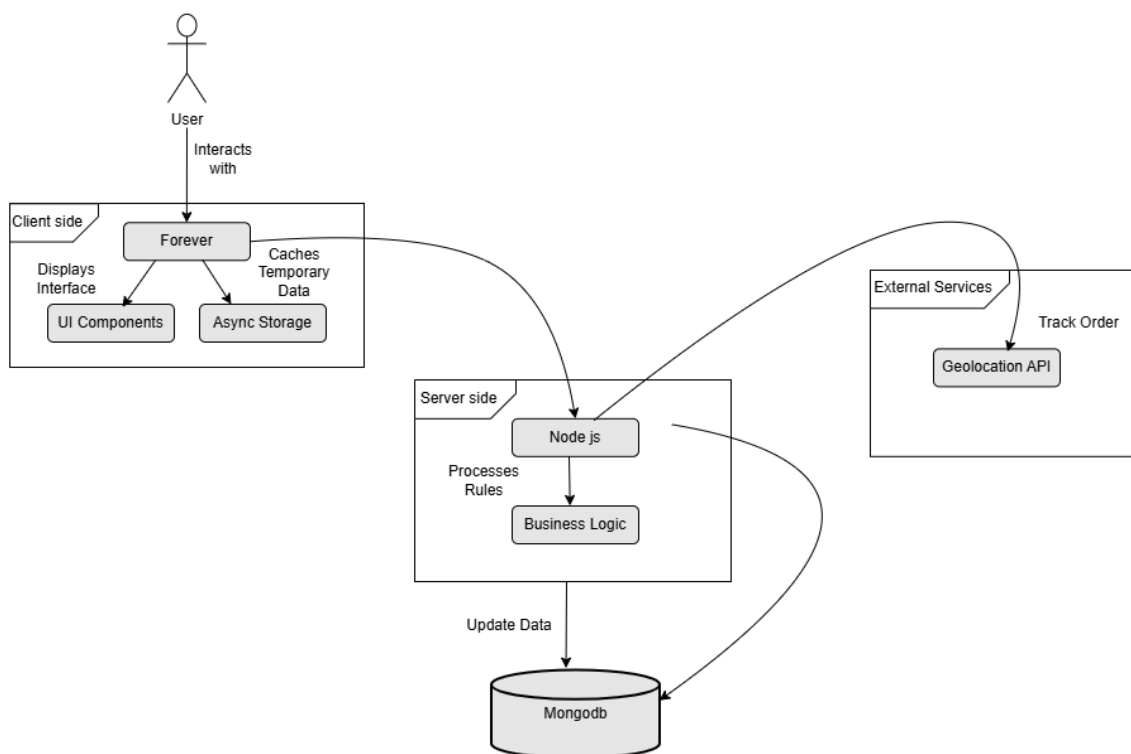


Figure 4. 1 Architecture Diagram of E-Garments “Forever.”

1. Overview of Components

The architecture comprises the following main sections:

- **Client Side**

The frontend interfaces the user interacts with.

- **Server Side**
Where the backend logic is handled, using **Node.js**.
- **Database**
A **MongoDB** database for storing application data.
- **External Services**
Third-party APIs for features like **order tracking**.

Each part is responsible for specific tasks, and their interaction ensures system functionality.

2. Client Side

a) User Interaction

- The system begins with the **User** who interacts with the application via a frontend interface called **E-Garments Forever**.
- The interface is responsible for accepting input (e.g., placing an order, browsing product) and displaying output (e.g., order status).

b) E-Garments Forever Interface

This is the core client-side component that facilitates user interaction. It includes:

- **UI Components**
 - These are the visible elements like buttons, forms, products, images, etc.
 - They provide the structure and visual feedback necessary for a seamless experience.
 - Examples include the "Place Order" button, order tracking display, or login form.
- **Async Storage**
 - This refers to browser-side storage used in web applications.
 - It stores user preferences, session tokens, or temporary cart data so the user doesn't lose progress if the app is refreshed or closed briefly.

c) Client Responsibilities

- Displaying the user interface.
- Accepting user input and forwarding it to the server.
- Temporarily caching data for better performance.
- Making API calls to fetch or send data.

3. Server Side

The **Server Side** is the backend that handles data processing and applies business rules.

a) Node.js

- The backend is built using **Node.js**, a JavaScript runtime known for handling asynchronous operations efficiently.
- Node.js receives requests from the frontend (e.g., new order, login, products fetch) and delegates them to the appropriate backend functions.

b) Business Logic

- This layer contains the rules and functions that govern how the system behaves.
- It processes the requests—e.g., validating user inputs, calculating delivery estimates, determining discounts, verifying user login credentials.
- After processing, it either returns the result directly or updates the database.

c) Interactions with MongoDB

- The processed data is stored or retrieved from the **MongoDB** database.
- Examples:
 - Saving a new order.
 - Retrieving Garment products.
 - Updating user details.
 - Checking order statuses.

d) Responsibilities of the Server

- Receiving and routing client requests.
- Validating and processing business logic.
- Communicating with the database.
- Calling external services when necessary.

4. Database Layer – MongoDB

- MongoDB is a **NoSQL database** used to store data in flexible, JSON-like documents.
- It is suitable for dynamic applications like **E-Garments "Forever"**, where data formats (e.g., different clothing products, sizes, colors, and user preferences) may vary.
- It is used for:
 - Storing **user data** (e.g., profile, addresses).

- Recording **orders** and their statuses.
- Managing **Garment data** (product, availability).
- Keeping **review and feedback** history.

a) Flow of Data

- Node.js fetches or updates data using MongoDB queries.
- The database returns the required information.
- Node.js formats the result and sends it to the frontend.

5. External Services

The architecture includes **External Services**, which the application calls for specialized functions that are not internally managed.

a) Track Order & Geolocation API

- A **Geolocation API** is integrated to provide real-time status of orders.
- This service fetches the current location of the delivery agent and allows customers to track the delivery status on a map.

6. How Components Work Together: Example Flow

Let's walk through a typical user journey to understand component interaction.

a) Step-by-Step Order Placement and Tracking

- **User opens the E-Garments Forever App**
 - Interacts with UI Components (Client Side).
- **User browses the product and places an order**
 - UI collects input and stores cart data in Async Storage.
 - Sends the order request to the Node.js backend.
- **Server processes the request**
 - Node.js checks business logic (e.g., is the garment open? is payment valid?).
 - Data is stored in MongoDB (new order entry).
- **Order is delivered**
 - Server updates MongoDB with delivery status.
 - User sees "Delivered" status on UI.

7. Key Benefits of This Architecture

The table 4.1 summarizes the system's key features, highlighting modular components, Node.js backend, MongoDB storage, external APIs, and asynchronous storage, each enhancing performance, scalability, and user experience.

Table 4. 1 Benefits of Architecture Diagram

Feature	Benefit
Modular Components	Easy to manage, update, and scale each part of the website independently (e.g., product catalog, user accounts, orders).
Node.js Backend	Fast processing of multiple requests using non-blocking I/O.
MongoDB Storage	Flexible and scalable data model ideal for dynamic user and order data.
External APIs	Real-time tracking and value-added services without building in-house.
Async Storage	Improves performance and resilience in poor network conditions.

8. Future Enhancements

To make this architecture more robust, the following can be considered:

- Introduce **Notification System** (SMS/email updates).
- Add an **Analytics Dashboard** for the admin to track sales, popular products, and customer behavior.

4.2 Data Representation [Diagram + Description]

The database contains multiple collections/tables

- **Users**
Stores login info, roles, and personal data.
- **Garments**

Product, profile, operating hours.

- **Products**

Clothing products including name, category, sizes, price, and availability.

- **Orders**

Customer orders and status.

- **Reviews**

Customer feedback on orders. Each entity is designed to normalize data and ensure integrity.

Entity Relationship Diagram

1. Introduction

An Entity Relationship Diagram (ERD) is a tool used to model the logical structure of a database. It defines the entities, their attributes, and the relationships between them. This ER diagram of E-Garment “Forever” (Figure 4.2) a comprehensive Customer Order and Delivery system, detailing the relationships between customers, orders, products, and delivery personnel. The core entities include Customer, Order, Product, and DeliveryBoy, showing a clear flow from customer placement to final delivery. A many-to-many relationship exists between Order and Product via the associative entity OrderProduct, capturing critical details like Quantity and Price. The diagram effectively represents the database schema for a system managing sales transactions, product inventory, and logistical tracking for delivery.

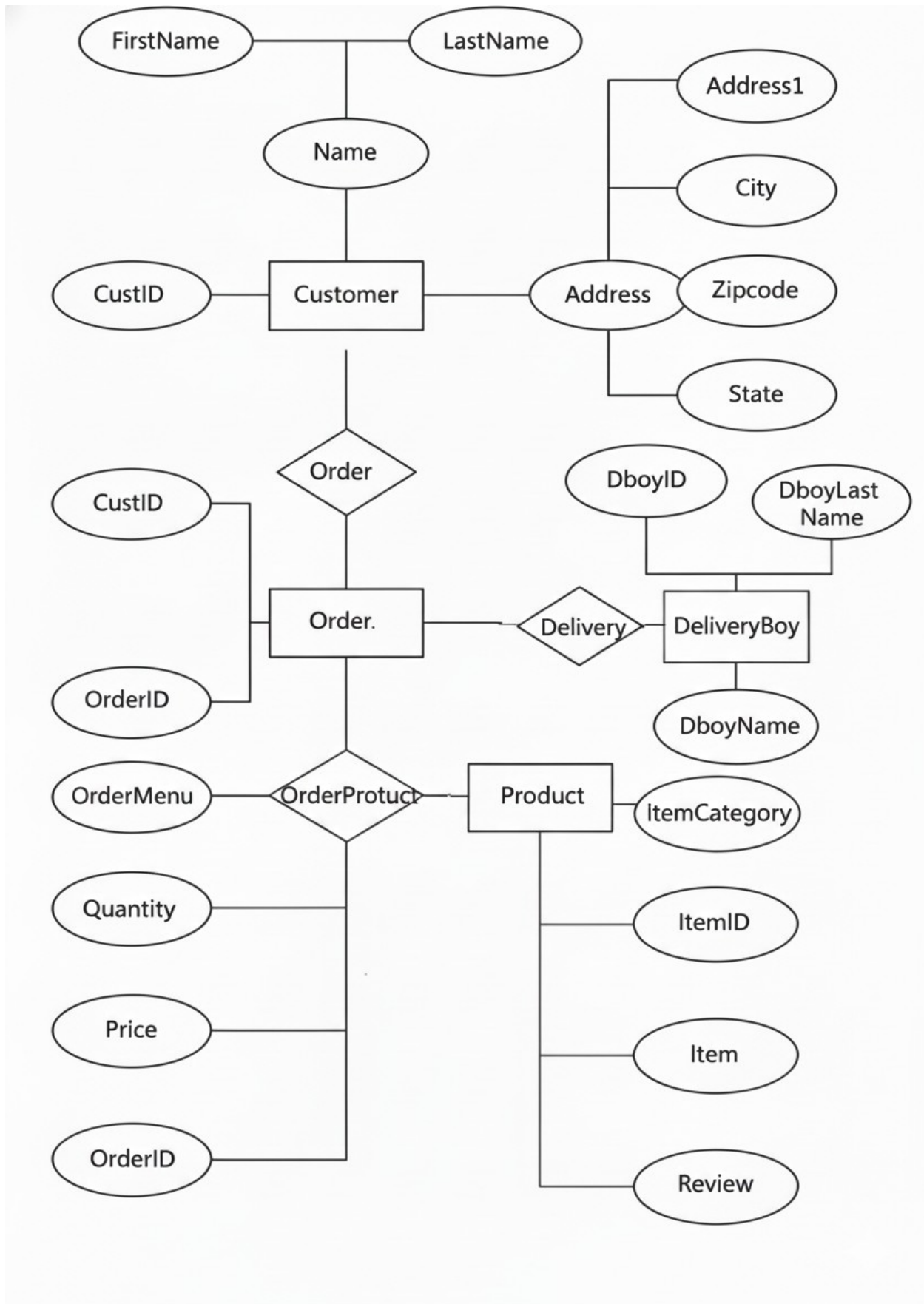


Figure 4. 2 ER Diagram of E-Garments "Forever"

2. Entity: Customer

The **Customer** is central to this system. This entity contains:

- **CustID**
A unique identifier for each customer. Acts as the primary key.
- **Name**
 - **FirstName**
 - **LastName**

The **Customer** entity is linked to:

- **Address** entity (to store where the order is to be delivered).
- **Orders** entity (indicating that each customer can place one or more orders).

3. Entity: Address

Each customer has an associated **Address**, broken down into:

- **Address1**
Street address or main address line.
- **City**
The city for the delivery.
- **Zipcode**
Postal code of the location.
- **State**
State or region.

This modular structure allows for precise and flexible address recording, which is crucial for delivery logistics.

4. Entity: Orders

The **Orders** entity represents each clothing order placed by customers. Key attributes include:

- **OrderID**
A unique identifier for each order.
- **CustID**
Foreign key that links back to the Customer entity.

The relationship is **one-to-many** a customer can place many orders, but each order belongs to only one customer.

The **Orders** entity connects to:

- **OrderProduct** (through which products are associated with the order).
- **Delivery** (to assign a delivery boy to the order).

5. Entity: OrderProduct

The **OrderProducts** is a junction table that links Orders with Products. An order can contain multiple products, and a product can appear in multiple orders.

Key attributes of **OrderProduct**:

- **OrderProductID**
Primary key.
- **OrderID**
Foreign key linking to the Orders table.
- **ProductID**
Foreign key linking to the Product table.
- **Quantity**
Number of units ordered for a particular products.
- **Price**
Price per unit at the time of ordering.

This structure allows for detailed breakdowns of each order.

6. Entity: Product

The **Product** entity stores information about all the Clothing products available in the system.

Attributes include:

- **ProductID**
Primary key for each product.
- **ProductName**
Name or description of the product.

- **ProductCategory**
E.g., Men's Wear, Women's Wear, Accessories.
- **Size**
Available sizes (S, M, L, XL).
- **Color**
Product color options.
- **Price**
The price at which the Product is sold.
- **Review**
A textual and numerical rating/review.

Each **Product** can be associated with multiple **OrderProduct** entries, reflecting sales across different orders.

7. Entity: DeliveryBoy

The **DeliveryBoy** entity handles the delivery personnel's data. Attributes include:

- **DboyID**
Primary key for each delivery boy.
- **DboyName**
Which is a composite of:
 - **DboyFirstName**
 - **DboyLastName**
- **DboyPhoneNo**
Contact number for delivery coordination.

Each order is assigned to one delivery boy, forming a **one-to-many** relationship between DeliveryBoy and Orders (one delivery boy can deliver multiple orders, but each order has one delivery boy).

8. Relationship: Delivery

- This is a relationship between **Orders** and **DeliveryBoy**, labeled **Delivery**.
- The **Delivery** relationship ensures that every order is linked with a delivery personnel responsible for fulfilling the delivery.

9. Relationships Summary

Relationships Summary of the E-Garments Forever (Table 4.2) summarizes the relationships between entities in the E-Garments Forever platform. It shows one-to-one, one-to-many, and many-to-one associations, such as a customer having one address, placing multiple orders, each order containing multiple products, each order product linked to a single product, and each order being delivered by one delivery boy.

Table 4. 2 Relationships Summary

Relationship	Entities Involved	Type	Explanation
Customer – Address	Customer, Address	1-to-1	A customer contains one address.
Customer – Orders	Customer, Orders	1-to-M	One customer can place multiple orders.
Orders – OrderProduct	Orders, OrderProduct	1-to-M	Each order can have multiple Product.
OrderProduct – Product	OrderProduct, Product	1-to-M	Each order Product entry is linked to a single product.
Orders – Delivery	Orders, DeliveryBoy via Delivery	1-to-M	Each order is delivered by one delivery boy.

10. Normalization

The design of this ERD reflects the principles of **database normalization**, especially **3NF (Third Normal Form)**:

- **No repeating groups**
Each piece of data is atomic.
- **No partial dependencies**

Every non-key attribute depends on the whole key.

- **No transitive dependencies**

non-key attributes only depend on the primary key.

This results in:

- Minimal data redundancy.
- Easier data maintenance.
- Improved consistency and data integrity.

11. Practical Scenario Walkthrough

Let's walk through a real-world example using this ERD.

Scenario: A customer places an order

- A new **Customer** is registered with **CustID**, **Name**, and **Address** (Address1, City, Zipcode, State).
- The customer places an **Order** (OrderID), linked to their CustID.
- The **OrderProduct** is filled with selected products, each linked to the **Product** through ProductID.
- The order contains:
 - Quantity of each product.
 - Price of each product at time of order.
- The customer may later submit a **Review** on the products from the **Product** entity.

12. Use in Application Development

This ERD can be translated into relational database tables during backend development. For instance:

- A **Customer** table with foreign keys referencing **Address**.
- A many-to-many join between **Orders** and **Product** using the **OrderProduct** table.

The clean design helps developers implement logic for:

- Order creation
- Delivery tracking
- Product browsing
- Review and rating systems

4.3 Process Flow/Representation

The Process flow diagram of E-Garments “Forever” (Figure 4.3) illustrates a platform for both garment owners and users. Garment owners can manage inventory, update orders, and check feedback, while users can register, browse items, add to cart, and complete payments. The system also supports order tracking, delivery updates, and feedback collection to ensure seamless interaction between users and garment owners.

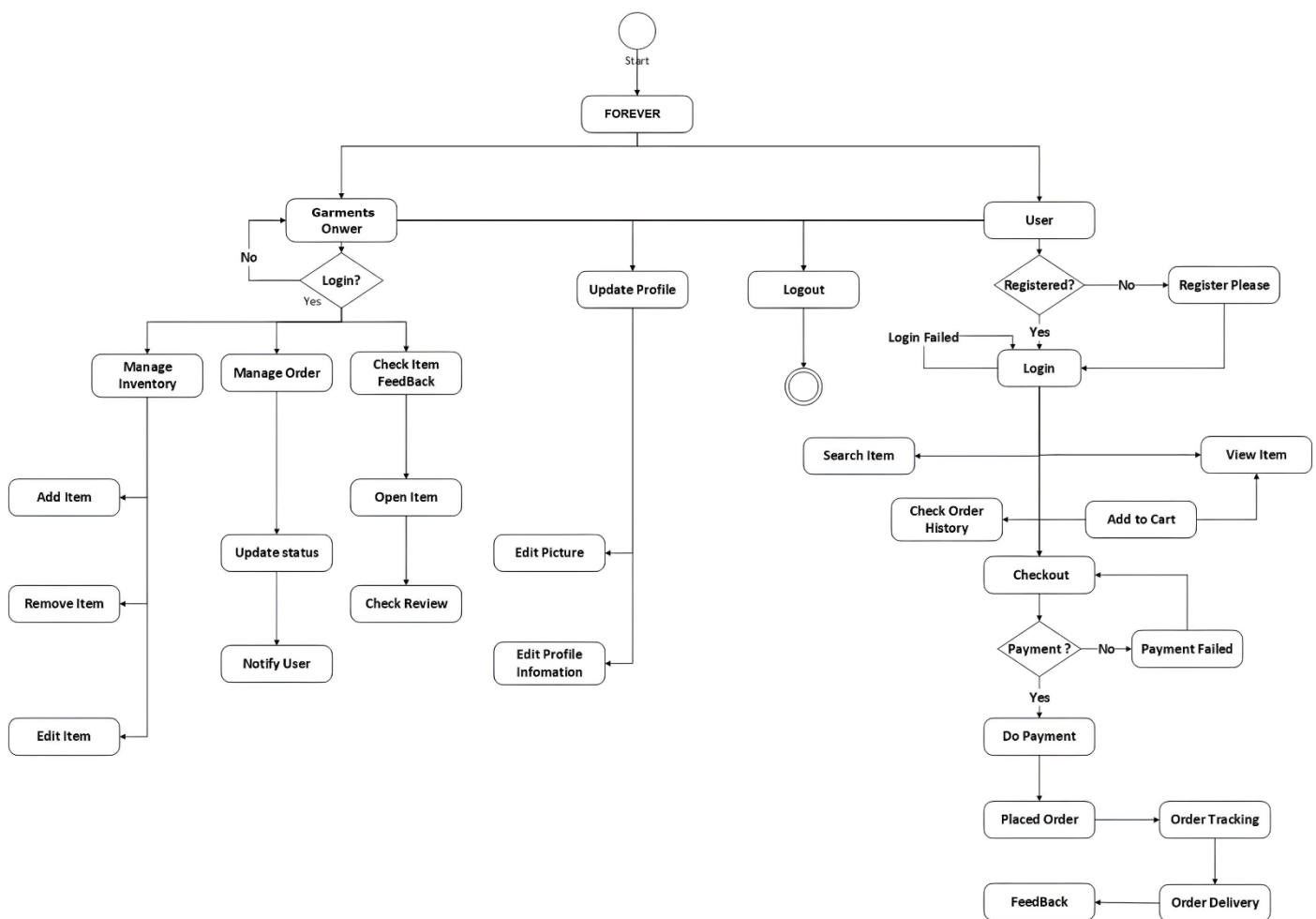


Figure 4. 3 Process flow of E-Garments “Forever”

1. Start of the System

The system initiates from the "Start" node and branches out into two major user types:

- Garments Owner
- User

Each branch then follows distinct workflows tailored to the needs and permissions of these user roles.

2. Garments Flow

Once a Garments owner enters the system, the first step is **Login**. If the login attempt is successful, the owner gains access to several functionalities. If the login fails, the flow simply halts until correct credentials are provided.

Upon successful login, the garments owner can:

a) Manage Inventory

- **Add Product**

Owners can add new clothing products or accessories to the website's product catalog.

- **Remove Product**

Products that are no longer available or out of stock can be removed from the inventory.

- **Edit Item**

Owners can modify the name, description, price, or availability of any existing Product.

This functionality ensures that the garments has full control over its offerings, reflecting real-time changes to the customer interface.

b) Manage Orders

- **Update Status**

The admin can update the current status of an order—such as "Processing", "Packed", "Shipped", or "Delivered".

This flow ensures clear communication between the garments and the customer, enhancing the user experience.

c) Check Product Feedback

- **Open Product**

Opens details of the product, including average ratings, sizes, colors, and stock availability.

- **Check Review**

Allows owners to view user-submitted reviews or feedback on individual products. This can help improve service quality and identify areas for improvement.

3. Customer Flow

This branch handles the user journey through the **E-Garments "Forever"** application. It begins with a **login /registration** phase and follows through the process of browsing, ordering, and reviewing.

a) User Registration and Login

- If the user is **not registered**, they are prompted with "Register Please" and are taken to the registration page.
- If already registered, they move to the **Login** phase.
- If **Login Fails**, the system asks for re-entry of credentials.

Only upon successful login can users access the system's main features.

b) Profile Management

- **Update Profile**
Users can upload or change their profile image.

c) Browsing and Searching

- **Search Product**
Users can search for clothing products or accessories based on keywords, categories, or filters.
- **Check Order History**
Displays the list of previous orders made by the user.
- **View Product**
Displays details about a selected product, including description, price, available sizes, and user ratings.

These features help users make informed decisions while ordering.

d) Cart and Checkout

- **Add to Cart**
Adds the selected products to the user's shopping cart.
- **Cart**
Shows all added products, including quantities, subtotal, and any applied discounts or taxes.
- **Checkout**
Proceeds to the payment and order confirmation page.

e) Payment Process

- **Payment**

A decision node checks if the payment was successful.

- **No → Payment Failed:** The system prompts the user to retry or use a different payment method.
- **Yes → Do Payment:** The payment is processed and confirmed.

Once the payment is successful, the order is considered placed.

f) Order Management

- **Placed Order**

Confirms that the order has been successfully submitted to the garments.

- **Order Tracking**

Allows users to track their order status in real-time.

- **Order Delivery**

Final step where the user receives the order.

This flow ensures a smooth and transparent experience from cart to delivery.

g) Feedback

- After the order is delivered, users are prompted to leave feedback.
- This feedback can include ratings, textual reviews, and is visible to both garments owners and other users.

4. Logout

Both users (garments owners and customers) can logout from the system once their tasks are completed. This returns them to a safe exit point and secures their session.

5. Flow Structure Overview

The flowchart is designed using standard flowchart symbols:

- **Terminator nodes** (rounded rectangles) for Start, Logout.
- **Decision nodes** (diamonds) for logical conditions (e.g., "Login?", "Registered?", "Payment?").
- **Processes** (rectangles) for each action, like "Add Product", "Search Product", "Checkout", etc.
- **Connectors/Arrows** to indicate the direction of flow.

Each component is logically connected, showing the dependencies between user actions and system responses.

4.4 Design Models

4.4.1 Sequence Diagram

The sequence diagram of E-Garments “Forever” (Figure 4.4) illustrates the order process of showing interactions between the customer, platform, database, garment admin, and payment system. Customers select products, add them to the cart, and proceed to checkout, while the system fetches product details and confirms orders. Payments are processed and verified, after which the order is confirmed and received by the customer.

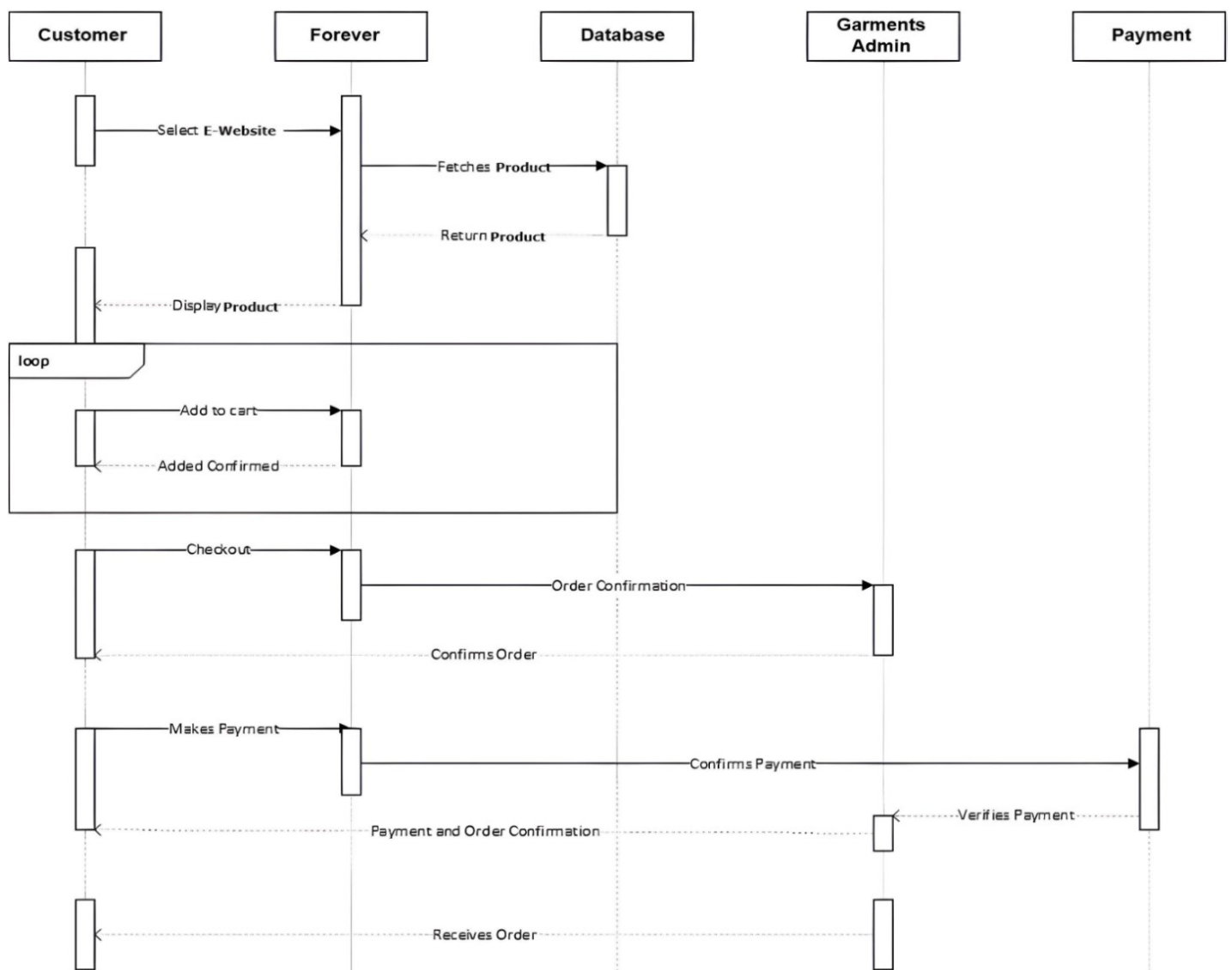


Figure 4. 4 Sequence Diagram of E-Garments “Forever”

1. Participants and Their Roles

a) Customer

The primary end-user who interacts with the application to browse garments, place an order, and complete payment..

b) Forever (System)

The central backend logic of the platform that handles customer requests, manages state, coordinates with the database.

c) Database

Stores persistent data, including user credentials garments product, order details, and transaction records.

d) Garments

Represents the vendor or store that manages the products, processes orders, and ensures timely dispatch and delivery to customers.

2. Sequence of Operations (Step-by-Step Explanation)

Step 1: Authentication Process

- **Customer → Forever:** Authentication
The customer initiates a login or authentication request.
- **Forever → Database:** Auth Request
The system sends the credentials (e.g., username, password) to the database to verify user identity.
- **Database → Forever:** Auth Successful
Once verified, the database returns a success response.

This authentication step ensures that the user is valid and authorized to use the system.

Step 2: Garments Selection and Products Retrieval

- **Customer → Forever:** Select garment
After login, the customer selects a product category (e.g., Men, Women, Accessories) or a specific store/brand.
- **Forever → Database:** Fetches Product
The system queries the database for product listings under the selected category or brand.

- **Database → Forever:** Return Product Listing

The database returns product details including name, size, price, colors, images, and availability.

- **Forever → Customer:** Display Product

The user interface displays the retrieved product list for browsing.

Step 3: Add to Cart

This step occurs inside a **loop**, meaning it can be repeated multiple times depending on how many products the customer wishes to add.

- **Customer → Forever:** Add to Cart

Each time a customer adds a product to the cart, this request is triggered.

- **Forever → Customer:** Added Confirmed

The system confirms that the selected product has been successfully added to the cart.

This iterative process allows customers to build their order before proceeding to checkout.

Step 4: Checkout and Order Confirmation

- **Customer → Forever:** Checkout

The customer initiates the checkout process once satisfied with their cart.

- **Forever → Garments:** Order Confirmation

The system sends the finalized order details to the admin for processing.

- **Garments Admin → Forever:** Confirms Order

The store acknowledges the order and begins packaging and dispatch preparation.

This step establishes the official start of the order fulfillment process on the garments end.

Step 5: Payment Processing

- **Customer → Forever:** Makes Payment

The customer provides payment details (COD).

- **Forever → Customer:** Order Confirmation

The system notifies the customer that the order has been confirmed.

Step 6: Order Delivery

- **Garments Admin → Customer:** Receives Order

The final stage occurs when the store dispatches and delivers the order to the customer through their delivery system or a partner service.

This completes the transaction lifecycle from browsing to delivery.

3. Features of the Diagram

a) Synchronous Communication

All the interactions shown are **synchronous calls**, indicated by solid lines with filled arrowheads. This means each message waits for a response before continuing, ensuring sequential integrity.

b) Object Lifelines

Each participant has a **lifeline** (dashed vertical line) that extends downwards, indicating their existence over the duration of the interaction.

c) Loop Notation

The loop symbol enclosing "Add to Cart" denotes that this action can happen multiple times dynamically, based on user behavior.

4. Importance in System Design

This sequence diagram plays a critical role in:

- **Clarifying communication protocols** between modules and external services. [5]
- **Validating system logic** for correctness and completeness.
- **Serving as documentation** for developers, QA engineers, and stakeholders.
- **Enhancing maintainability**, especially during debugging or system updates.

5. Summary of User Experience Flow

- User logs in securely.
- Browses through the garment categories and selects a specific vendor or brand.
- Views product details such as images, sizes, colors, and price.
- Selects one or multiple products and adds them to the cart.
- Proceeds to checkout to confirm delivery details and make payment.
- Receives order updates after vendor confirmation until the product is delivered.

This flow is designed for **efficiency**, **clarity**, and **security**, ensuring a smooth user experience while also accommodating backend processes.

4.4.2 Class Diagram

The Class diagram of E-Garments “Forever” (Figure 4.5) illustrates the main system classes with their attributes and methods, showing how core components User, Garments, ProductItem, Order, and Review are interconnected. These relationships define data flow and module interaction, supporting key functions like user management, product browsing, order processing, and customer feedback.

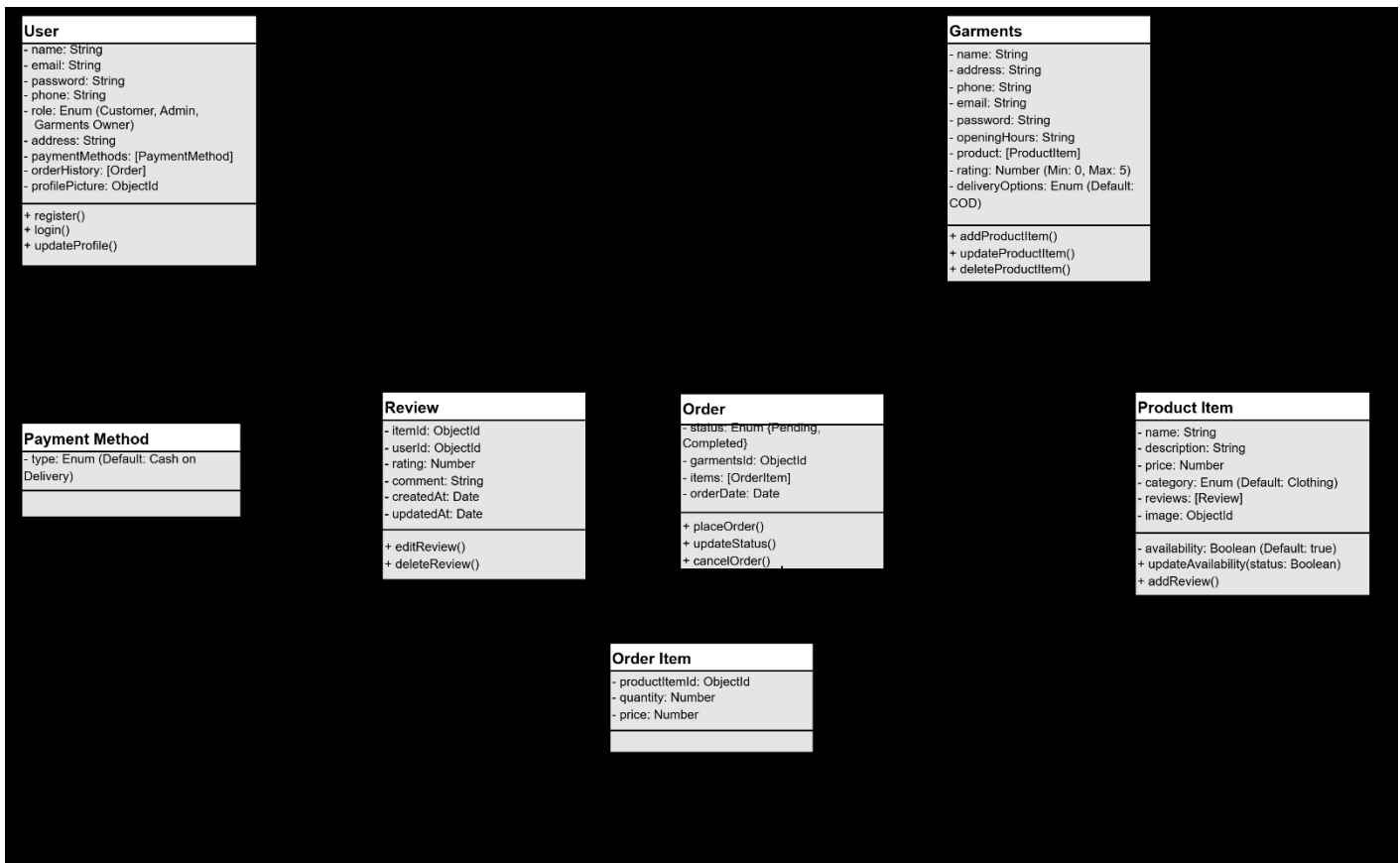


Figure 4. 5 Class Diagram of E-Garments “Forever”

1. User Class

a) Attributes:

- Name
The full name of the user.
- Email
Email ID used for login and communication.
- Password

Encrypted password for account access.

- Phone
Contact number.
- Role
Enum type that distinguishes between Customer, Admin, and Garments Owner.
- Address
User's delivery address.
- orderHistory
A list of orders made by the user.
- profilePicture
Reference to a stored image for the user's avatar.

b) Methods:

- register()
Used to create a new user account.
- login()
Authenticates the user into the system.
- updateProfile()
Allows updating user details (like name, address, etc.).

c) Relationships:

- **A User has many Payment Methods** (1 to many).
- **A User has many Orders** (1 to many).
- **A User can write many Reviews** (1 to many).

This class is central to the system as it supports multiple user types, enabling different roles to access tailored functionalities.

2. PaymentMethod Class

a) Attributes:

- Type
Enum indicating payment method, defaulting to Cash on Delivery.

b) Relationships:

- Connected to User class in a one-to-many relationship.

It allows each user to have payment method saved, facilitating faster and more flexible checkout.

3. Garments Class

a) Attributes:

- name, address, phone, email, password: Basic credentials and contact details.
- openingHours
String indicating business hours.
- Rating
Numeric rating (0 to 5) based on user reviews.
- deliveryOptions
Cash on Delivery

b) Methods:

- addProductItem()
Allows a garments vendor to add a new product listing.
- updateProductItem()
Updates details of an existing product.
- deleteProductItem()
Removes a product from the catalog.

c) Relationships:

- One **Garments** vendor can have many **ProductItems** (1-to-many).
- One **Garments** vendor can receive and process many **Orders** (1-to-many).

The **Garments** class represents the vendors providing clothing and accessories within the “Forever” system. It includes functionality for managing product inventories and handling customer orders.

4. ProductItem Class

a) Attributes:

- name, description, price: Basic details about the product.

- **Category**
Enum representing product type (e.g., Men, Women, Accessories).
- **Reviews**
Linked to Review objects.
- **Image**
Link or ID for the item image.
- **Availability**
Boolean indicating if the item is currently available.
- **updateAvailability(status)**
Method to toggle availability.
- **addReview()**
Allows customers to submit reviews for a product.

b) Relationships:

- A ProductItem is linked to exactly **one Garments** (many-to-one).
- A ProductItem can have **many Reviews** (1 to many).

This class is essential for representing the catalog of products available for purchase, enabling dynamic updates and user interaction.

5. Order Class

a) Attributes:

- **Status**
Enum—Pending, Completed, or others (e.g., Canceled).
- **Garments**
Reference to the vendor fulfilling the order.
- **Products**
List of **OrderItem** objects selected by the customer.
- **orderDate**
Date the order was placed.
- **placedOrder()**

Function to initiate order creation.

- `updateStatus()`
For order tracking (e.g., Accepted, Preparing, Delivered).
- `cancelOrder()`
Enables order cancellation before a defined status.

b) Relationships:

- One Order is placed by one **User**.
- One Order contains **many OrderItems** (1 to many).
- One Order is assigned to one/multiply **Garments**.

The Order class acts as a bridge between the user and the Garments, representing all key transactional information.

6. OrderItem Class

a) Attributes:

- `ProductItemId`
Reference to the selected `ProductItem`.
- `Quantity`
Number of units ordered.
- `Price`
Price at the time of order (useful for audit/history).

b) Relationships:

- Belongs to one Order.
- References one `ProductItem`.

This class captures individual products within an order, supporting accurate billing and purchase history tracking.

7. Review Class

a) Attributes:

- `itemId`
References the product item being reviewed.

- `userId`
Reviewer identity.
- `Rating`
Numeric score, typically between 1 and 5.
- `Comment`
Optional written feedback.
- `createdAt, updatedAt`
Timestamps for the review.
- `deleteReview()`
Function to remove the review.

b) Relationships:

- A Review links a User to a ProductItem.
- Many reviews can be written for the same ProductItem.

Reviews help in quality assurance, transparency, and user decision-making.

8. Entity Relationships Summary

Here's a quick overview of cardinalities:

Class Diagram Relationship of the E-Garments Forever (Table 4.3) indicating the cardinality between classes, such as one-to-one between User and PaymentMethod, one-to-many between User and Order or Review, and many-to-one between ProductItem and OrderItem.

Table 4.3 Class Diagram Relationship

Relationship	Cardinality
User → PaymentMethod	1 to 1
User → Order	1 to M
User → Review	1 to M
garment → ProductItem	1 to M
Garments → Order	1 to M
ProductItem → Review	1 to M

Order → OrderItem	1 to M
ProductItem → OrderItem	M to 1

9. Use Cases and Real-World Mapping

- A customer logs into the system, browses a garments vendor's product catalog, and adds selected products to the cart.
- When the customer places an order, an **Order** object is created containing multiple **OrderItem** entries, each representing a selected product.
- Payment details are fetched from the user's PaymentMethod list.
- After receiving the order, the customer may submit a **Review** for each purchased ProductItem.
- Garments vendors can update their ProductItems, manage availability, and monitor all incoming Orders.

This tightly linked structure ensures referential integrity and supports scalable implementation

Chapter 5

Implementation

5 Implementation

The implementation of the E-Garments Forever system involved building a full-stack application composed of multiple modules, each designed to serve a specific user group customers, garments owners, and backend administrators. This chapter outlines the major components of the implementation phase, focusing on how the backend, web interfaces were developed and integrated into a cohesive system.

The project was developed using the Incremental Software Development Model, which allowed for iterative development and early integration of core functionalities. This approach ensured flexibility in adapting the design and logic over time based on real-time testing and user feedback during implementation. The entire system is built on modern JavaScript technologies:

- **Backend**

Node.js with Express.js and MongoDB (Mongoose) for RESTful API development.

- **Customer Web Portal**

Developed using React.js for users preferring a browser-based interface.

5.1 Backend Implementation

The backend serves as the core service layer of the system. It is responsible for handling authentication, business logic, data persistence, and communication between the frontend clients and the database.

Key implementation details include:

- **Authentication**

JSON Web Tokens (JWT) are used for secure authentication of both users and garments. Passwords are hashed using bcrypt before storage for enhanced security.

- **Routing & Controllers**

A modular structure was implemented, separating routes, controllers, models, and middleware. This ensures clean code organization, maintainability, and scalability.

- **Database Design**

MongoDB collections are structured to support dynamic documents. Key collections include Users, Garments, ProductItems, Orders, Reviews, and Images.

- **File Uploads**

Images are uploaded to the server and linked with Products or garments profiles. Unique UUIDs are used to prevent filename conflicts and ensure uniqueness.

5.2 Customer Web Portal Implementation

The customer web portal is a web-based interface designed for desktop and laptop users, providing full access to all system functionalities. It is developed using React.js and communicates with the backend API, ensuring consistent business logic and real-time data handling across the platform.

Implemented components include:

- **Home Page**

Displays a catalog of garments and stores with search and filtering options to enhance user navigation and product discovery.

- **Product View**

Allows users to explore detailed product information, including images, sizes, colors, prices, and availability, and add products directly to their shopping cart.

- **Checkout Flow**

Offers a streamlined process for cart review, order confirmation, and payment processing.

- **Responsive Design**

Implements responsive layouts to ensure usability and optimal viewing across various screen sizes and devices.

5.3 User Interface

The UI of E-Garments “Forever” was designed with a focus on user experience (UX) and responsiveness. The system adapts to different screen sizes (desktop, tablet, and mobile).

5.3.1 Web Interface

Login Page

The login page of the E-Garments Forever (Figure 5.1) platform provides a simple and intuitive interface for users to access their accounts. The mockup includes fields for entering an email and password, along with a secure authentication process that validates user credentials before granting access. A “Login” button is prominently displayed, and additional options such as “Forgot Password?” and “Create Account” ensure easy navigation and account recovery for

users. This design ensures a clean, user-friendly experience that supports efficient and secure entry into the system.

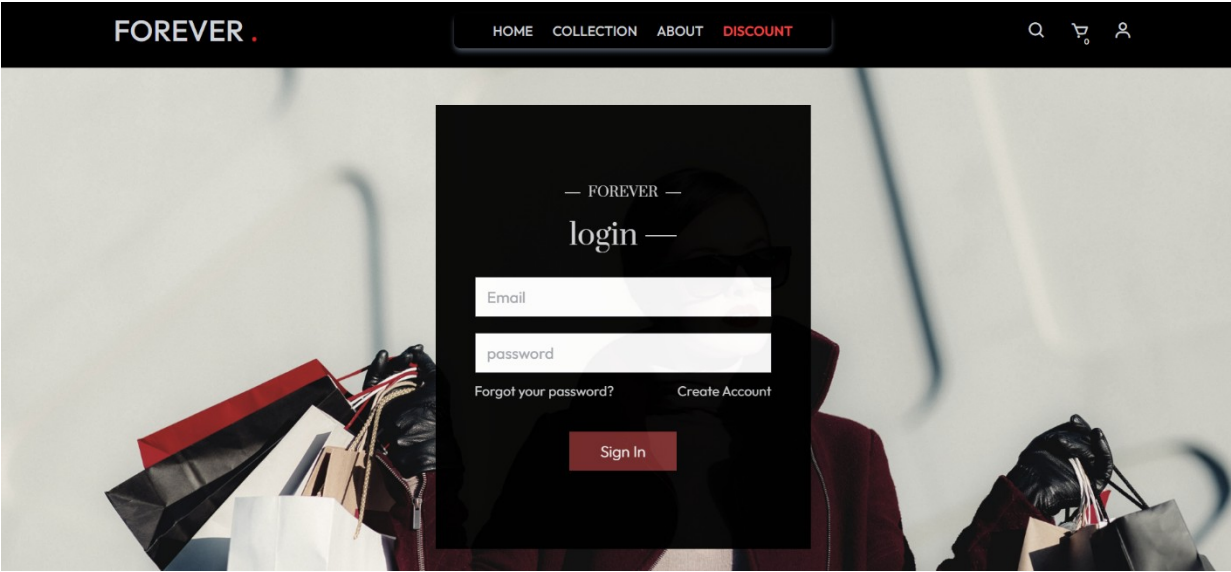


Figure 5. 1 Login Page of E-Garments “Forever”

Home Page

The home page of the E-Garments Forever (Figure 5.2) platform provides a visually clean and intuitive layout, acting as the central access point for users. The top navigation menu includes four main options Home, Collection, About and Discount allowing customers to easily browse products, learn about the platform, and explore ongoing offers. The mockup also features highlighted product categories, promotional banners, and a search bar, enabling users to quickly navigate the platform and discover relevant items with ease.

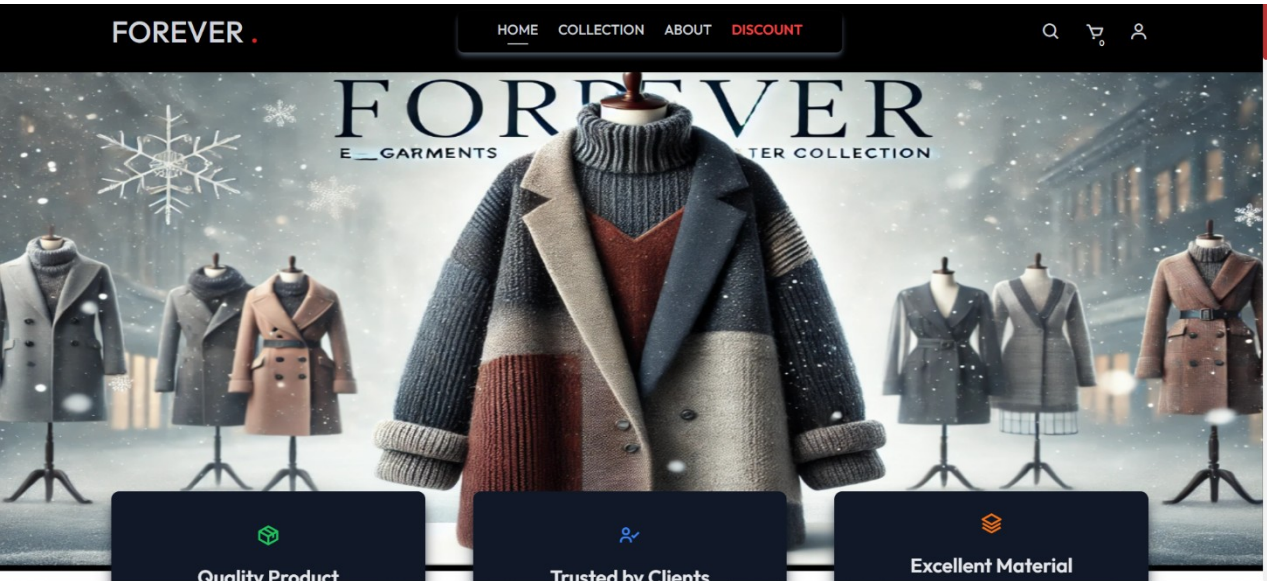


Figure 5. 2 Home Page of E-Garments “Forever”

Products Page

The products page of the E-Garments Forever (Figure 5.3) platform displays all available clothing items in a structured and visually organized manner. The mockup presents products in a grid layout, showing essential details such as item name, price, image, and stock availability. Users can navigate through categories, apply filters, and use the search option to refine their choices. This page ensures a smooth browsing experience by allowing customers to quickly compare items and select products based on their preferences.

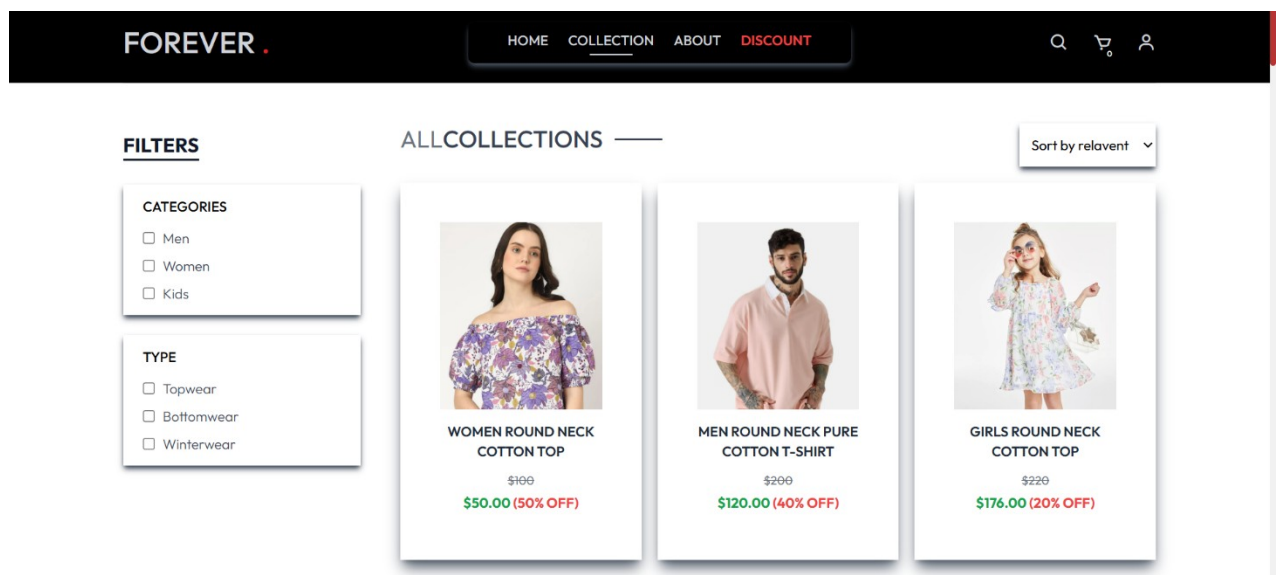


Figure 5. 3 Products Page of E-Garments “Forever”

Admin Dashboard Page

The Admin Dashboard page of the E-Garments Forever (Figure 5.4) platform provides a centralized interface for managing the entire system efficiently. The mockup presents key modules such as product management, order tracking, inventory updates, and customer activity monitoring. Visual elements like summary cards, charts, and quick-access menus allow the admin to view real-time statistics, manage garments or user operations, and handle ongoing tasks with ease. This dashboard ensures streamlined administration and supports effective decision-making across the platform.

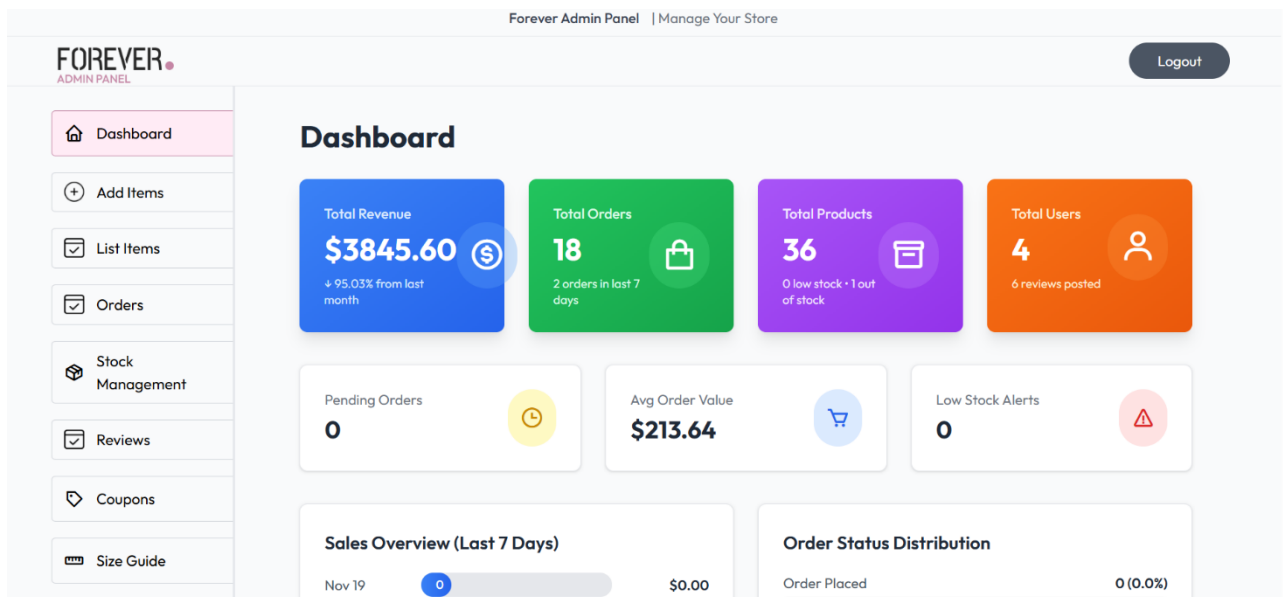


Figure 5. 4 Admin Dashboard Page of E-Garments “Forever”

Cart Page

The Cart page of the E-Garments Forever (Figure 5.5) platform provides users with a clear summary of the products they intend to purchase. The mockup displays each selected item with its image, name, price, quantity selector, and subtotal, allowing users to review and modify their cart before checkout. A detailed cost breakdown including item total, delivery charges, and final payable amount is shown, along with a “Proceed to Checkout” button to guide users smoothly toward order placement. This page ensures transparency, accuracy, and a seamless shopping experience.

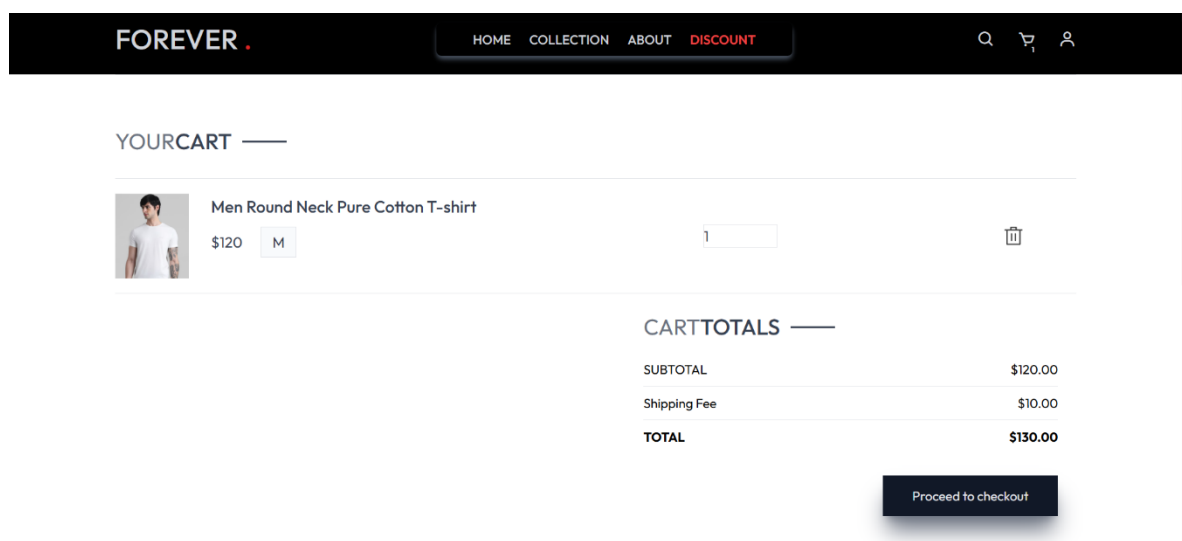


Figure 5. 5 Cart Page of E-Garments “Forever”

Stock Management Page

The Stock Management page of the E-Garments Forever (Figure 5.6) platform enables garment owners to efficiently monitor and update their product inventory. The mockup presents a structured table displaying each item's name, price, current stock, minimum stock level, and status indicators for low or out-of-stock items. Options such as "Update Stock," "Edit Item," and "Add New Product" allow vendors to make quick adjustments and keep inventory accurate. This page ensures real-time visibility of product availability, supporting smooth operations and preventing stock shortages.

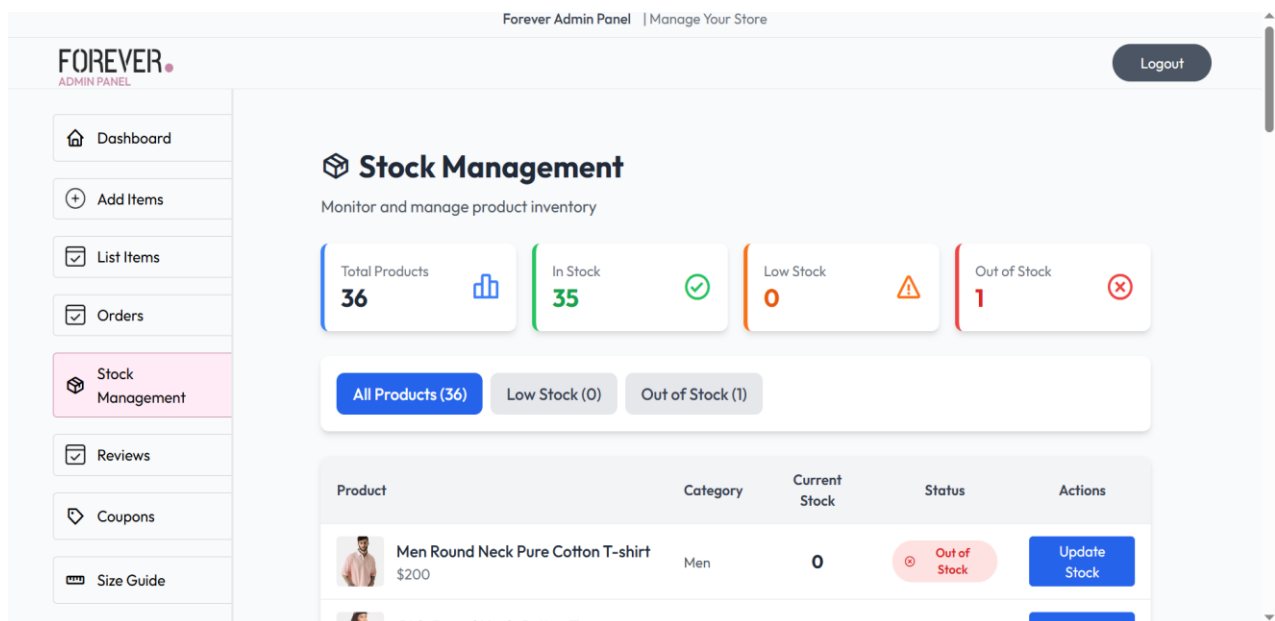


Figure 5. 6 Stock Management Page of E-Garments "Forever"

Delivery Detail Page

The Delivery Detail page the E-Garments Forever (Figure 5.7) provides a clear and organized view of the delivery information associated with a customer's order. The mockup includes the customer's name, address, phone number, estimated delivery time, and order status, ensuring accuracy and transparency throughout the delivery process. Additional elements such as delivery notes, assigned delivery personnel, and real-time status updates help streamline the fulfillment process and improve customer experience.

FOREVER.

HOME COLLECTION ABOUT DISCOUNT

Q 🛒 👤

DELIVERYINFORMATION

First name

Last name

Email address

Street

City name

State name

Zip code

Country

Phone

CARTTOTALS

Have a Coupon Code?

ENTER CODE

Apply

SUBTOTAL

\$120.00

Shipping Fee

\$10.00

Total

\$130.00

PAYMENTMETHOD

☐ stripe

☐ Razorpay

☒ CASH ON DELIVERY

PLACE ORDER

Figure 5. 7 Delivery Detail Page of E-Garments “Forever”

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

This chapter include the following sections.

- Manual Testing
- Automated Testing

Manual Testing

Manual testing was conducted to ensure that all individual components and integrated modules performed correctly according to the requirements.

6.1.1 System testing

System testing validated the overall functionality of the system in a real-world usage scenario. This ensured that all integrated components (user interface, backend APIs, database, and third-party APIs) worked together seamlessly.

- **Objective**

Confirm that the entire application operates as intended.

- **Scope**

Includes full workflows: registration, login, ordering, order tracking, and review submission.

- **Result**

All test scenarios passed without major defects.

6.1.2 Unit Testing

Each functional unit was tested individually to verify its correctness. Below are examples:

- **Unit Testing 1**

Login Module

- **Objective**

Ensure valid login credentials grant access, and invalid ones are rejected.

In Unit Test 1 of E-Garments Forever (Table 6.1) shows listing test cases for login functionality, including valid login, invalid password, and empty fields, along with expected outcomes and test results, demonstrating the system's handling of different login scenarios.

Table 6. 1 Unit Test 1

No.	Test Case	Input	Expected Result	Result
1	Valid login	Username: Ali, Pwd: 123	Dashboard loads	Pass
2	Invalid password	Username: Ali, Pwd: xyz	Error: Incorrect password	Fail
3	Empty fields	-	Error: Fields cannot be empty	Fail

- **Unit Testing 2**

Edit Profile

- **Objective**

Ensure profile updates reflect accurately in the system.

In Unit Test 2 of E-Garments Forever (Table 6.2) shows testing profile update functionality, including updating the phone number and handling empty required fields, with expected outcomes and pass/fail results.

Table 6. 2 Unit Test 2

No.	Test Case	Input	Expected Result	Result
1	Update phone number	New number entered	Profile updated successfully	Pass
2	Leave required field empty	Remove email	Error: Required field missing	Fail

6.1.3 Functional Testing

Functional testing ensured that the system fulfilled all functional requirements listed in the SRS.

- **Functional Testing**

Order Placement

- **Objective**

Verify that a customer can place an order and receive confirmation.

In Functional Test of E-Garments Forever (Table 6.3) showing test cases for adding products and checkout, as well as handling an empty cart, with expected outcomes and pass/fail results.

Table 6. 3 Functional Test

No.	Test Case	Input	Expected Result	Result
1	Add products and checkout	Selected 3 products, filled address	Order confirmed and ID generated	Pass
2	Attempt to place empty cart	No products selected	Error: Cart is empty	Fail

6.1.4 Integration Testing

Integration testing was conducted to ensure proper data flow between system components.

In Integration Testing of E-Garments Forever (Table 6.4) shows the integration test results for the E-Garments Forever platform, demonstrating scenarios like placing an order, updating order status, and submitting a review, along with the expected outcomes and pass results, verifying smooth interaction between different system modules.

Table 6. 4 Integration Test

No.	Scenario	Input	Expected Outcome	Result
1	Login to place order	User logs in, selects products and checks out	Order confirmation, garments receives order	Pass
2	Garments receives and marks order prepared	Admin marks status as 'Ready for delivery'	Customer receives live status update	Pass
3	Submit a review after delivery	Customer rates item	Review saved; average rating updated	Pass

6.2 Automated Testing:

Although most testing was manual, a few automated tests were implemented using tools such as Postman.

Tool Used of E-Garments Forever (Table 6.5) showing that Postman was used for API testing on login, registration, and order placement, with successful results.

Table 6. 5 Tool Used

Tool	Description	Applied On	Results
Postman [6]	API testing tool	Login, Register, Place Order APIs	Pass

Chapter 7

Conclusion and Future Works

7 Conclusion and Future Work

Summarizes project outcomes and proposes future directions.

7.1 Conclusion

The E-Garments “Forever” project successfully delivers a full-stack, web-based e-commerce platform for garments and accessories. The system is scalable, user-friendly, and responsive, allowing customers to browse products, place orders, and manage profiles seamlessly. Vendors are provided with an intuitive interface to manage their product catalog, track orders, and view customer feedback.

From requirement analysis and system design to implementation and testing, every phase was executed following established software engineering practices, particularly the Incremental SDLC model. The platform meets all major functional and non-functional requirements outlined in the SRS and has been validated through comprehensive manual and automated testing.

Overall, E-Garments “Forever” achieves its goal of providing a modern, flexible, and localized e-commerce solution that addresses limitations found in existing online garment platforms.

7.2 Future Work

While the current version of E-Garments “Forever” is robust and fully functional, several enhancements can be implemented in future iterations to improve performance, scalability, and user experience:

- **Delivery/Logistics Module**

A separate module or application can be developed to manage delivery personnel, track shipments, and update order statuses in real time. This ensures timely delivery of products and improves overall customer satisfaction.

- **Payment Gateway Integration**

Add secure online payment options to provide more flexibility.

- **AI-Based Recommendations**

Implement recommendation systems to suggest meals based on user preferences or order history.

- **Push Notifications**

Enable real-time notifications for order status changes, promotions, and reviews.

8. References

- [1] Y. Z. e. al, "Cross-platform mobile application development with React Native: A performance study," *IEEE Access*, vol. 8, pp. 111011-111021, 2020.
- [2] Z. W. e. al, "Real-time delivery tracking systems for e-commerce logistics," *International Journal of Transportation Science and Technology*, vol. 8, no. 4, pp. 365-376, 2019.
- [3] Node.js, "Node.js Documentation," 2023. [Online]. Available: <https://nodejs.org/>.
- [4] B. M. a. R. Gupta, "Evaluating Node.js for scalable web applications," *Journal of Web Development*, vol. 15, pp. 45-52, 2019.
- [5] Norman, "The Design of Everyday Things," *MIT Press*, 2013.
- [6] Postman, "API Testing Tool," 2023. [Online]. Available: <https://www.postman.com/>.
- [7] M. Fowler, *UML distilled: a brief guide to the standard object modeling language*, Addison-Wesley Professional, 2004.
- [8] MongoDB, "MongoDB Documentation," 2023. [Online]. Available: <https://www.mongodb.com/docs/>.
- [9] E. a. T. J. B. Foster, *Software engineering: a methodical approach*, Auerbach Publications, 2021.
- [10] Shopify. (2024, March 15). What is Inventory Management? How to Manage Your Fashion Inventory: <https://www.shopify.com/blog/inventory-management-for-fashion>